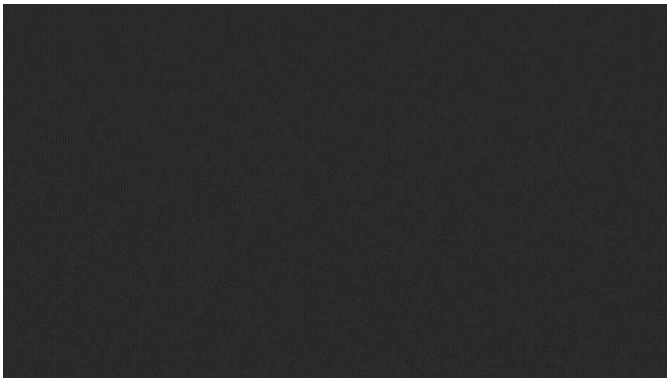
Portenta Hat Carrier User Manual

Learn about the hardware and software features of the Arduino® Portenta Hat Carrier.

Author • Taddy Chung Last revision • 09/25/2024

Overview

The user manual offers a detailed guide on the Arduino Portenta Hat Carrier, consolidating all its features for easy reference. It will show how to set up, adjust, and assess its main functionalities.



This manual will show the user how to proficiently operate the Portenta Hat Carrier, making it suitable for project developments related to industrial automation, manufacturing automation, robotics, and prototyping.

Hardware and Software Requirements

Hardware Requirements

To use the Portenta Hat Carrier, it is necessary to attach one of the boards from the Portenta Family:

- Arduino Portenta X8
- Arduino Portenta C33
- Arduino Portenta H7

Additionally, the following accessories are needed:

- USB-C® cable (x1)
- Wi-Fi® Access Point or Ethernet with Internet access (x1)

Software Requirements

If you want to use the Portenta Hat Carrier with a Portenta X8, check the following bullet points:

- Make sure you have the latest Linux image. Refer to this section to confirm that your Portenta X8 is up-to-date.

> To ensure a stable operation of the Portenta Hat Carrier with Portenta X8, the minimum Linux image version required for Portenta X8 is **746**. To flash the latest image on your board, you can use the Portenta X8 Out-of-the-box or flash it manually downloading the latest version directly from this link.

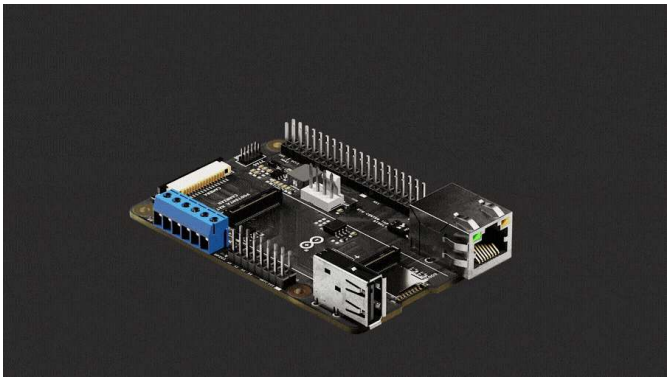
- Arduino IDE 1.8.10+, Arduino IDE 2.0+, or Arduino Cloud Editor in case you want to use the auxiliary microcontroller of the Portenta X8 to run Arduino code.

In case you want to use the Portenta Hat Carrier with a Portenta H7/C33:

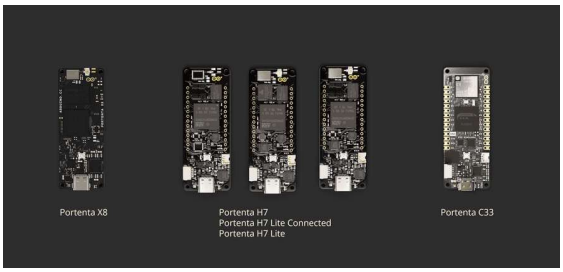
- Arduino IDE 1.8.10+, Arduino IDE 2.0+, or Arduino Cloud Editor is needed to use the Portenta H7/C33 to run Arduino code.

Product Overview

The **Portenta Hat Carrier** offers a platform for developing a variety of robotics and building automation applications. It provides access to multiple peripherals, including CAN FD, Ethernet, microSD, and USB, as well as a camera interface via MIPI and 8x analog pins. On the other hand, the dedicated debug pins for JTAG and the PWM fan connector help simplify your Portenta applications.



The carrier is adaptable, pairing seamlessly with Portenta X8 and converting it into an industrial Linux platform compatible with Raspberry Pi® Hats. It also works with Portenta H7 and Portenta C33, providing versatile solutions to meet demands across various requirements.



Compatible Portenta boards

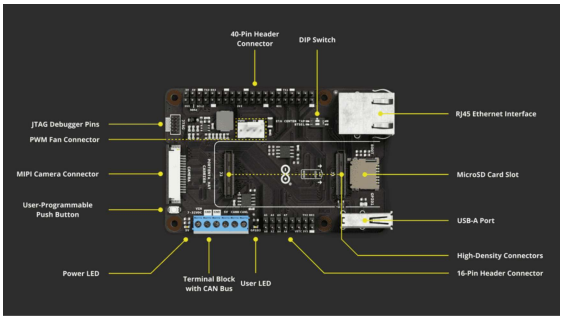
Carrier Architecture Overview

The **Portenta Hat Carrier**, designed for Portenta SOM boards like the Portenta X8, H7, and C33, offers a diverse power supply range:

- 7-32V through its screw terminal
- USB-C®
- 5V pin on the 40-pin header

This versatility extends to its connectivity: a USB-A for peripherals, 1000 Mbit Base-T Ethernet, SPI, I2C, I2S, and UART interfaces accessible via a 40-pin male header, and MIPI camera support exclusive for the Portenta X8.

It integrates a microSD slot for storage and data logging, broad interface options through its 40-pin and 16-pin headers, JTAG pins for debugging, and a PWM fan connector for cooling. The Ethernet speed control is intuitive with a two-position DIP switch, allowing various profiles based on the paired Portenta board.



Portenta Hat Carrier board overview

The Portenta Hat Carrier has the following characteristics:

Compatible SOM boards: The carrier is compatible with: Portenta X8 (ABX00049), Portenta H7 (ABX00042/ABX00045/ABX00046) and Portenta C33 (ABX00074).

Power management: The board can be powered up from different sources. The onboard screw terminal block allows a 7-32 V power supply to power the Portenta board and

The USB-C® interface of the Portenta X8, H7, and C33 can supply the needed power to the board. Alternatively, the 5V pin from the 40-pin male header can be used to power the board. The carrier can deliver a maximum current of 1.5 A.

USB connectivity: A USB-A female connector is used for data logging and the connection of external peripherals like keyboards, mice, hubs, and similar devices.

Communication: The carrier supports Ethernet interface (X1) via RJ45 connector 1000 Base-T connected to High-Density pins (J1). If paired with Portenta H7 or C33, the maximum speed is limited to 100 Mbit Ethernet.

The SPI (X1), I2C (x2), I2S (x1), and UART (x2) are accessible via a 40-pin male header connector. The I2C1 is already dedicated to the EEPROM memory but is accessible through a 40-pin male header connector on SCL2 and SDA2.

The UARTs do not have flow control, and UART1 and UART3 can be accessed via a 40-pin connector while UART2 can be accessed via a 16-pin connector. The **CAN** (x1) bus is available with an onboard transceiver. The **MIPI** camera is also available but only when the Portenta X8 is attached. Examples of compatible devices include the OmniVision OV5647 and the Sony IMX219 sensors.

Storage: The board has a microSD card slot for data logging operation and bootloading operation from external memory.

Ethernet connectivity: The carrier offers a Gigabit Ethernet interface through an RJ45 connector 1000 Base-T. If the carrier is paired with Portenta H7 or C33, the maximum speed is limited to 100 Mbit Ethernet.

40-pin male header connector: The connector allows for SPI (x1), I2S (x1), SAI (x1), 5V power pin (x2), 3V3 power pin (x2), I2C (x2), UART (x2), PWM pins (x7), GND (x8), and GPIO (X26). The I2C count includes the one that is dedicated to EEPROM. UARTs do not have flow control. The GPIO pins are shared with different functionalities.

16-pin male header connector: The connector allows analog pins (x8), PWM (x2), LICELL (x1), GPIO (x1), 3V3 (x1), GND (x1), serial TX (x1), and serial RX (x1).

Screw terminal block: The terminal block allows power supply line feed for the carrier and bus ports. It consists of VIN 7 ~ 32 VDC (x1), VIN 5 V (x1), CANH (x1), CANL (x1), and GND (x2).

Debug interface: The carrier features an onboard 10x pin 1.27mm JTAG connector.

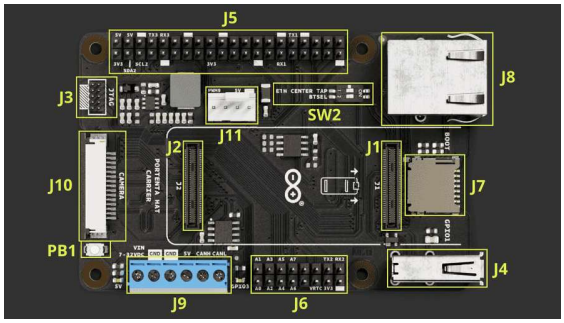
PWM fan connector: The board has an onboard PWM fan connector (x1) compatible with a 5 V fan with a PWM signal for speed regulation.

DIP switch: The carrier features a DIP switch with two positions, allowing for different profiles depending on the paired Portenta board. This DIP switch includes both the ETH CENTER TAP and BTSEL switches.

The ETH CENTER TAP controls the Ethernet interface. The OFF position enables Ethernet for the Portenta X8. Conversely, the ON position enables Ethernet for the Portenta H7/C33.

The BTSEL switch can be used to set the Portenta X8 into Flashing Mode when the switch is set to the ON position.

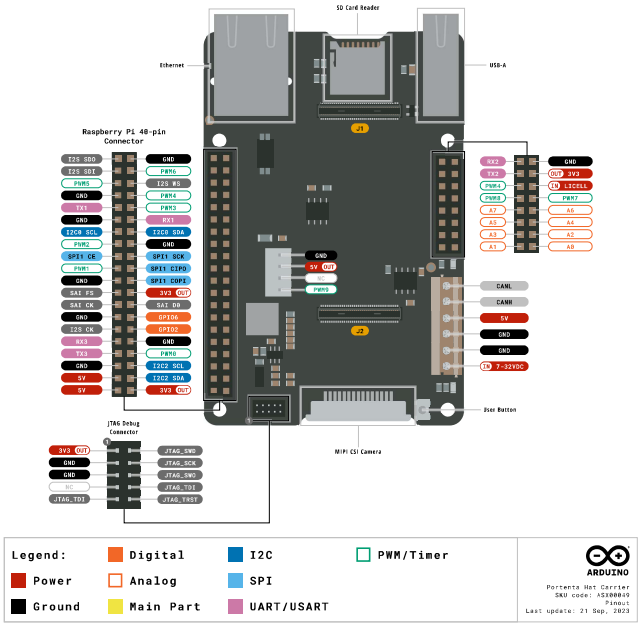
Carrier Topology



Portenta Hat Carrier Topology

Item	Onboard modules
J1, J2	High-Density connectors for Portenta boards
J3	JTAG male connector for debugging
J4	USB-A female connector for data logging and external devices
J5	40-pin male header compatible with Raspberry Pi® Hats
J6	16-pin male header for analog, GPIOs, UART and RTC battery pins
J7	MicroSD slot for data logging and media purposes
J8	RJ45 connector for Ethernet
J9	Screw terminal for power supply and CAN FD support
J10	MIPI camera connector, exclusive for Portenta X8
J11	PWM male header connector to control fan speed
SW2	DIP switch with two sliders: *ETH CENTER TAP* and *BTSEL*
PB1	User Button

Pinout



Datasheet

The full datasheet is available and downloadable as PDF from the link below:

Portenta Hat Carrier Datasheet

Schematics

The full schematics are available and downloadable as PDF from the link below:

Portenta Hat Carrier Schematics

STEP Files

The full *STEP* files are available and downloadable from the link below:

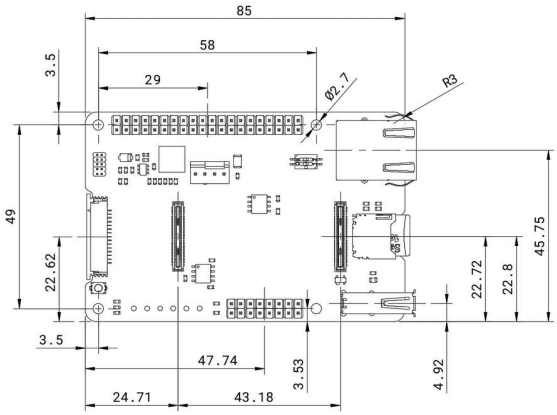
Portenta Hat Carrier STEP files

Mechanical Information

In this section, you can find mechanical information about the Portenta Hat Carrier. The dimensions of the board are all specified here, within top and bottom views, including the placements of the components onboard.

If you desire to design and manufacture a custom mounting device or create a custom enclosure for your carrier, the following image shows the dimensions for the mounting holes and general board layout. The given dimensions are all in

You can also access the STEP files which are also available here.

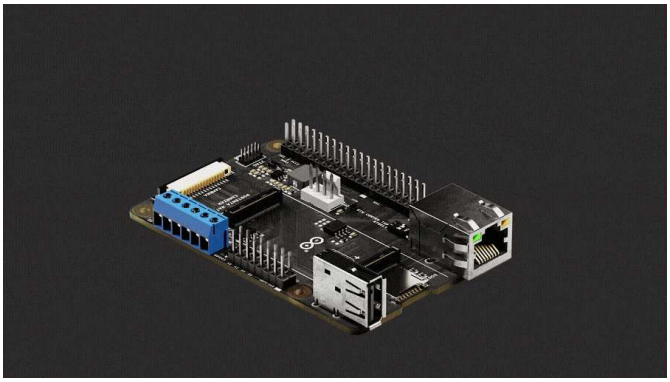


Portenta Hat Carrier Overall Dimensions

First Use Of Your Portenta Hat Carrier

Stack The Carrier

The Portenta Hat Carrier design allows the user to easily stack the preferred Portenta board. The following figure shows how the Portenta Hat Carrier pairs with the Portenta boards via High-Density connectors.



With the Portenta mounted to the carrier, you can proceed to power the carrier and start prototyping.

Power The Board

The Portenta Hat Carrier can be powered according to one of the following methods:

- Using an **external 7 to 32 V power supply connected to the VIN pin available on the screw terminal block of the board is the most recommended method**. It ensures that the Portenta Hat Carrier, the SOM, and any connected

For clarity on the connection points, please refer to the board pinout section of the user manual. Ensure the supplied current meets the specification for all components, as shown in the operating conditions table reported later on.

Using an external **5 V power supply** to:

The *5V* pin located on the 40-pin male header connector pins

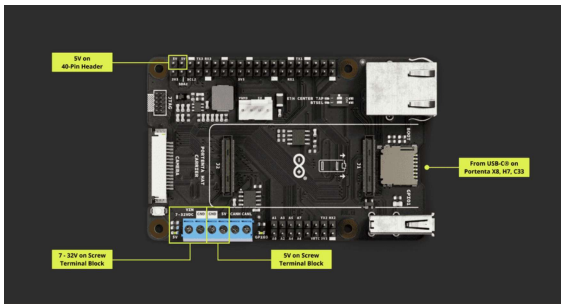
The *5V* pin located on the screw terminal of the board

You can effectively power the Portenta Hat Carrier, the SOM, and any connected hat.

For more details on this connection, kindly consult the board pinout section of the user manual. Again, ensure that the power supply's maximum current respects all components' specifications.

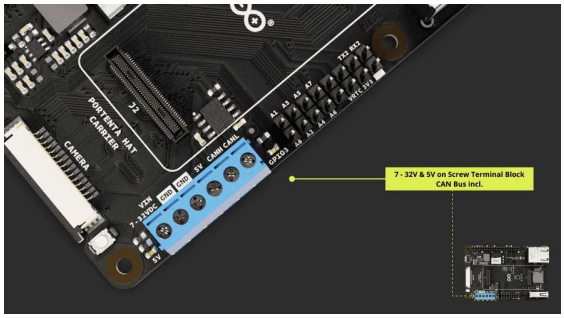
Using a USB-C® cable (not included) connected to the Portenta core board of your choice powers not only the selected core board, like the Portenta X8, H7, or C33, but also the Portenta Hat Carrier, and any connected hat that does not require a dedicated external power supply.

The Portenta Hat Carrier can deliver a **maximum** of 1.5 A.



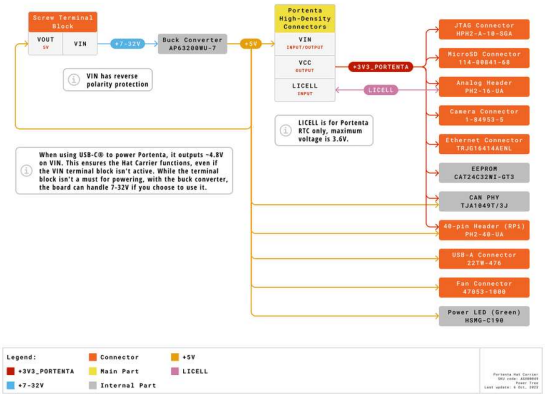
Portenta Hat Carrier Power Connection Overview

The image below magnifies the location of the terminal block for the 7 - 32 V and 5 V power source of the Portenta Hat Carrier:



Portenta Hat Carrier Power Source within Terminal

Subsequently, you can check how the Portenta Hat Carrier distributes power resources with the power tree diagram.



Portenta Hat Carrier Power Tree Diagram

Recommended Operating Conditions

To ensure the safety and longevity of the board, it is essential to understand the carrier's operating conditions. The table below provides the recommended operating conditions for the carrier:

Parameter	Min	Typ	Max	Unit
VIN from onboard screw terminal* of the Carrier	7.0	-	32.0	V
USB-C® input from the connected Portenta family board	-	5.0	-	V
+5 VDC from the 40-pin header connector on the carrier	-	5.0	-	V
+5 VDC from the carrier's onboard screw terminal	-	5.0	-	V
Current supplied by the carrier	-	-	1.5	A
Ambient operating temperature	-40	-	85	°C

The onboard screw terminal powers both the carrier and any connected Portenta board. Additionally, this terminal connector includes reverse polarity protection for enhanced safety.

Carrier Characteristics Highlight

The Portenta Hat Carrier provides different functionalities based on the connected Portenta family board, as shown in the table below:

Features	Portenta X8	Portenta H7	Portenta 33
40-Pin	Compatible	Compatible	Compatible

Features	Portenta X8	Portenta H7	Portenta 33
16-Pin Header	Compatible	Compatible	Compatible
MIPI Camera	Compatible	Incompatible	Incompatible
Ethernet	1 Gbit (DIP: OFF)	100 Mbit (DIP: ON)	100 Mbit (DIP: ON)
PWM Fan	Available	Available	Available
JTAG Debug	Available	Available	Available
Storage Exp.	MicroSD slot	MicroSD slot	MicroSD slot
CAN FD	Available	Available	Available
USB-A Support	Available	Available	Available

The Portenta X8 is the specific Portenta family board that offers compatibility with Raspberry Pi® Hats on the 40-pin Header.

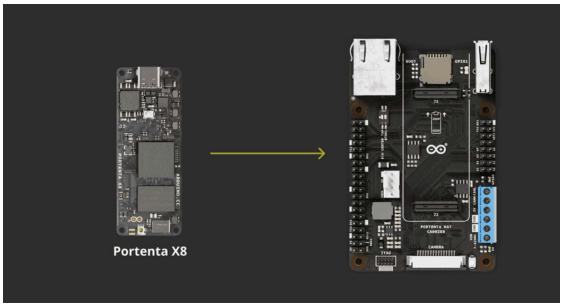
This provides a general idea of how the Portenta Hat Carrier will perform depending on the paired Portenta board. Each feature is explained in the following section after a quick guide covering how to properly interface the Portenta boards.

Hello World Carrier

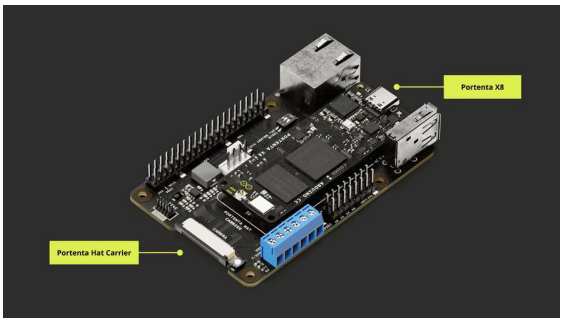
Hello World Using Linux

Using Portenta X8 with Linux

To use the Portenta Hat Carrier with the Portenta X8, you will have to align the High-Density connectors along with the USB-C® port. The following diagram shows how the board stacks on the carrier.



Portenta Hat Carrier with X8



Portenta Hat Carrier with X8

For the stable functionality of the Portenta Hat Carrier when used with Portenta X8, it is crucial to have at least version **746** of the Linux image on the Portenta X8. Access and download the latest version directly through this **link**.

Hello World With Portenta X8 Shell

A series of *Hello World* examples will be used to ensure the Portenta Hat Carrier is correctly operating with the paired Portenta X8. These examples, using Linux commands, Python® scripts, and the Arduino IDE, aim to trigger the user-programmable LED connected to GPIO3 leveraging different methods and platforms.

We will begin with a *Hello World* example using Linux commands. The user-programmable LED can be controlled using commands within the Portenta X8's shell. Learn how to connect with the Portenta X8 shell **here**.

The following commands will help you set and control the GPIO3, which connects to the user-programmable LED.

Let us begin with the commands to access the Portenta X8's shell:

```
adb shell
sudo su -
```

When you execute the *sudo su -* command, you will be prompted for a password:

The default password is `fio`

This command grants you access as the root user, loading the root user's environment settings such as `$HOME` and `$PATH`.

The aforementioned commands allow you to access the Portenta

Subsequently, use the following command to export the gpio device located under `/sys/class/`. In this context, *GPIO3* corresponds to *GPIO 163*, which is associated with the user-programmable LED we aim to access.

```
echo 163 > /sys/class/gpio/export
```

Using the following commands will help you verify the available GPIO elements.

```
ls /sys/class/gpio
```

It lists all the GPIOs previously initialized by the system. Meanwhile, the following command lists the details of *GPIO 163*, corresponding to *GPIO3*, which was previously imported:

```
ls /sys/class/gpio/gpio163
```

The GPIO can now be configured verifying that GPIO3 elements were successfully exported. The following command with the `I/O` field will set the I/O state of the pin. The pin can be set either as Input using `in` or Output using `out` value.

```
echo <I/O> >/sys/class/gpio/gpio163/direction
```

For this example, we will replace the `<I/O>` field with `out` value within the following command:

```
echo out >/sys/class/gpio/gpio163/direction
```

To verify the pin setting, use the `cat` command. If correctly configured, this command will display the set value:

```
cat /sys/class/gpio/gpio163/direction
```

The GPIO is now set as an output, thus it can now be controlled

To set the pin High, you need to assign the value `1`, or `0` to set the pin `HIGH` or `LOW`. The command will require the `value` at the end to ensure the pin's state is controlled.

To set the pin to `HIGH`:

```
echo 1 >/sys/class/gpio/gpio163/value
```

To set the pin to `LOW`:

```
echo 0 >/sys/class/gpio/gpio163/value
```

If you have finished controlling the GPIO, you can use the following command to *unexport* it, ensuring it no longer appears in the userspace:

```
echo 163 >/sys/class/gpio/unexport
```

To confirm that the specified GPIO has been properly unexported, you can use the following command:

```
ls /sys/class/gpio
```

This step helps you to prevent unintentional modifications to the element configuration.

Hello World Using Linux and Python® Scripts

Previously, we manually toggled the LED linked to *GPIO3* on the Portenta X8 via the command line. However, to automate this process and potentially extend our control logic, we can employ a Python® script for this purpose.

The script below is compatible with the ADB shell on the Portenta X8:

```python
#!/usr/bin/env python3
import time

class GPIOController:
    def __init__(self, gpio_number):
        self.gpio_number = gpio_number
        self.gpio_path = f"/sys/class/gpio/gp

    def export(self):
        with open("/sys/class/gpio/export", "
            f.write(str(self.gpio_number))

    def unexport(self):
        with open("/sys/class/gpio/unexport",
            f.write(str(self.gpio_number))

    def set_direction(self, direction):
        with open(f"{self.gpio_path}direction
            f.write(direction)

    def read_direction(self):
        with open(f"{self.gpio_path}direction
            return f.read().strip()

    def set_value(self, value):
        with open(f"{self.gpio_path}value", "
            f.write(str(value))
```

The script can be named `hello_world_python.py` for example. It can then be pushed to Portenta X8 using the following command on a computer terminal:

```
adb push hello_world_python.py /home/fio
```

The file is uploaded to the `/home/fio` directory. Navigate to the directory using ADB shell:

```
cd python3 hello_world_python.py
```

Now use the following command to run the script:

```
python3 hello_world_python.py
```

Portenta Hat Carrier's user-programmable LED will start blinking whenever the script is running.

Please check out the **Portenta X8 user manual** to learn how the

Portenta Hat Carrier. The Portenta Hat Carrier supports the Portenta X8 via High-Density connectors.

The Portenta X8 has the capability to operate in a Linux environment and it is based on Yocto Linux distribution. It is recommendable to read how the Portenta X8 works in terms of Linux environment here.

Hello World Using Arduino

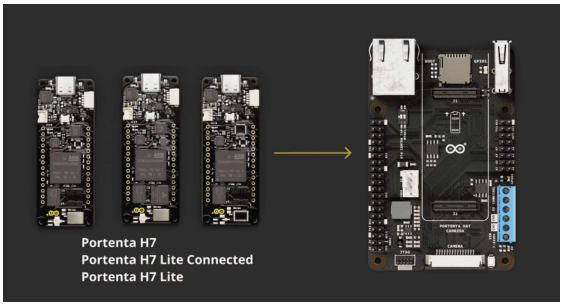
Using Portenta X8 / H7 / C33 with Arduino

The Portenta X8 is also capable of operating within the Arduino environment and retains the same hardware setup as explained here.

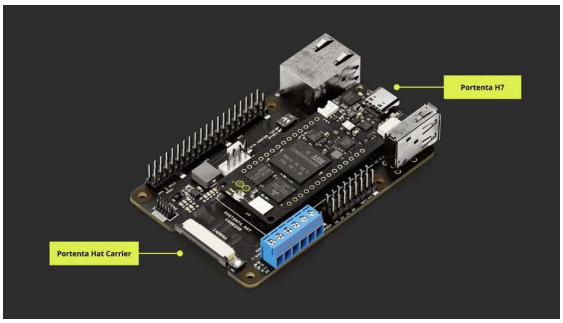
The Portenta H7 and C33 boards have hardware setups similar to the Portenta X8. To mount them on the Hat Carrier, please align the High-Density connectors along with USB-C® port orientation.

The diagrams below show how the Portenta H7 and C33 stack on the carrier:

Portenta H7

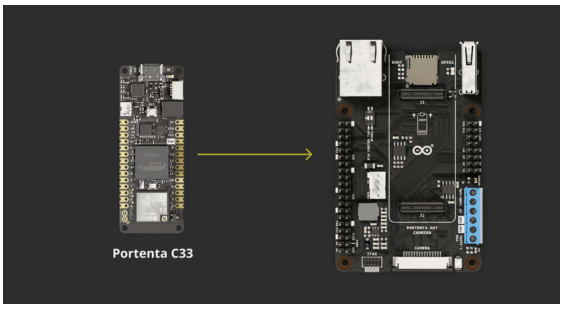


Portenta Hat Carrier with H7

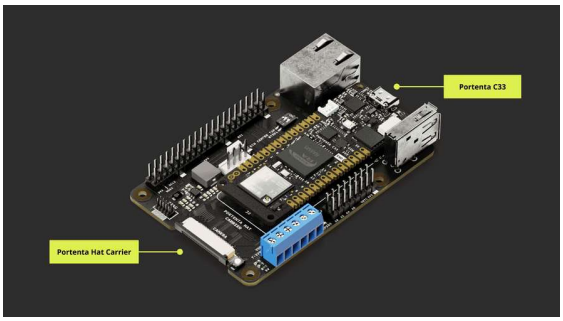


Portenta Hat Carrier with H7

Portenta C33



Portenta Hat Carrier with C33



Portenta Hat Carrier with C33

Hello World With Arduino

In this section, you will learn how to use the Portenta X8, Portenta H7, or Portenta C33 with the Portenta Hat Carrier. You will interact with the user-configurable LED connected to GPIO3, but this time within the Arduino environment.

Once any compatible Portenta board is connected to the Portenta Hat Carrier, launch the Arduino IDE 2 and set up the subsequent sketch:

```
// the setup function runs once when you press
void setup() {
  // Initialize the digital pin of the chosen S
  pinMode(<DIGITAL_PIN>, OUTPUT);
}

// the loop function runs over and over again f
void loop() {
  digitalWrite(<DIGITAL_PIN>, HIGH);         //
  delay(1000);                               //
  digitalWrite(<DIGITAL_PIN>, LOW);          //
  delay(1000);                               //
}
```

Make sure to replace `<DIGITAL_PIN>` with the appropriate value for your chosen Portenta board:

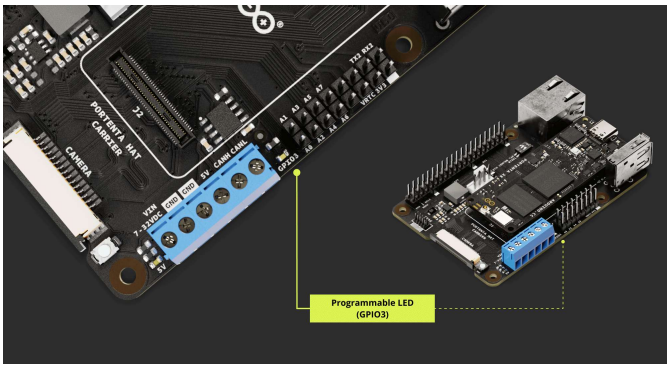
Portenta H7: PD_5

Portenta C33: 30

For example, when using the Portenta X8, your script should look like this:

```
// the setup function runs once when you press
void setup() {
  // Initialize the digital pin of the chosen S
  pinMode(PF_4, OUTPUT);
}

// the loop function runs over and over again f
void loop() {
  digitalWrite(PF_4, HIGH);         // turn the
  delay(1000);                      // wait for
  digitalWrite(PF_4, LOW);          // turn the
  delay(1000);                      // wait for
}
```

After successfully uploading the sketch, the user-configurable LED will start blinking. The following clip illustrates the expected LED blink pattern.



Please check out the following documentation to learn more about each board and maximize its potential when paired with the Portenta Hat Carrier:

- Portenta C33 user manual.
- Portenta H7 set-up guide.
- Portenta X8 user manual. You can also read the tutorial providing a step-by-step guide on how to upload sketches to the M4 Core on Arduino Portenta X8 here.

Please note that the Ethernet connectivity speed is limited to 100 Mbit when used with the Portenta H7 or C33.

For up-to-date performance of the Portenta X8 on the Portenta Hat Carrier, ensure you update to the latest Portenta X8 OS image. You can check here for more details.

Carrier Features and Interfaces

The carrier offers a diverse range of features and interfaces to cater to a variety of user requirements and applications. This section provides an overview of the main hardware interfaces, storage options, and configuration mechanisms integrated into the carrier.

Each sub-section further delves into the specifications of each feature, ensuring users will get comprehensive information for optimal utilization.

Hardware Interfaces

This sub-section introduces the essential hardware connection points and interfaces present on the Portenta Hat Carrier. Ranging from connectors and camera interfaces to fan control, you will be able to explore the physical interaction points of the carrier.

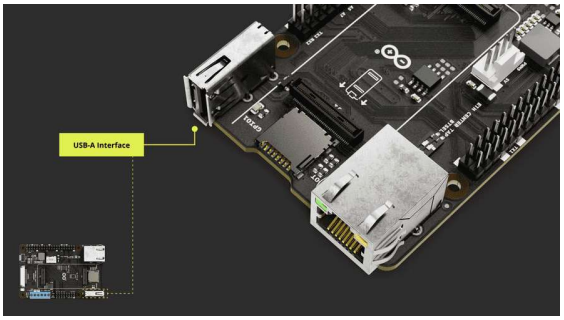
High-Density Connectors

The Portenta X8, H7, and C33 enhance functionality through High-Density connectors. For a comprehensive understanding of these connectors, please refer to the complete pinout documentation for each Portenta model.

- Complete Portenta X8 pinout information
- Complete Portenta H7 pinout information
- Complete Portenta C33 pinout information

USB Interface

The Portenta Hat Carrier features a USB interface suitable for data logging and connecting external devices.



Portenta Hat Carrier USB-A Port

If you are interested in the USB-A port pinout, the following table may serve to understand its connection distribution:

Pin number	Power Net	Portenta HD Standard Pin	High-Density Pin	Interface
1	+5V	VIN / USB0_VBUS	J1-21, J1-24, J1-32, J1-41, J1-48	
2		USB0_D_N	J1-28	USB D-
3		USB0_D_P	J1-26	USB D+
4	GND	GND	J1-22, J1-31, J1-42, J1-47, J1-54, J2-24, J2-33, J2-44, J2-57, J2-70	

Devices with a USB-A interface, such as storage drives, can be used for logging data. External devices include peripherals like keyboards, mouses, webcams, and hubs.

Using Linux

As an example, the following command on Portenta X8's shell can be used to test a write command with a USB memory drive. To write a file, the following sequence of commands can help you to accomplish such task.

```
sudo su -
```

First of all, let's enter root mode to have the right permissions to mount and unmount related peripherals like our USB memory drive.

```
lsblk
```

The `lsblk` command lists all available block devices, such as hard drives and USB drives. It helps in identifying the device name, like `/dev/sda1` which will be probably the partition designation of the USB drive you just plugged in. A common trick to identify and check the USB drive connected is to execute the `lsblk` command twice; once with the USB disconnected and the next one to the USB connected, to compare both results and spot easily the newly connected USB drive. Additionally, the command `lsusb` can be used to gather more information about the connected USB drive.

```
mkdir -p /mnt/USBmount
```

The `mkdir -p` command creates the directory `/mnt/USBmount`. This directory will be used as a mount point for the USB drive.

```
mount -t vfat /dev/sda1 /mnt/USBmount
```

This mount command mounts the USB drive, assumed to have a FAT filesystem (`vfat`), located at `/dev/sda1` to the directory `/mnt/USBmount`. Once mounted, the content of the USB drive can be accessed from the `/mnt/USBmount` directory with `cd`:

```
cd /mnt/USBmount
```

Now if you do an `ls` you can see the actual content of the connected USB Drive.

```
ls
```

Let's create a simple text file containing the message `Hello, World!` in the already connected USB memory drive using the following command:

```
dd if=<(echo -n "Hello, World!") of=/mnt/USBmour
```

This command uses the `dd` utility, combined with process substitution. Specifically, it seizes the output of the `echo` command, responsible for generating the `Hello, World!` message, and channels it as an input stream to `dd`.

Subsequently, the message gets inscribed into a file named *helloworld.txt* situated in the `/mnt/USBmount` directory.

After creating the file, if you wish to retrieve its contents and display them on the shell, you can use:

```
cat helloworld.txt
```

This command `cat` prompts in the terminal the content of a file, in this case the words `Hello, World!`.

expand the possibilities of your solution with the additional storage connected to the Portenta Hat Carrier.

Using Arduino IDE

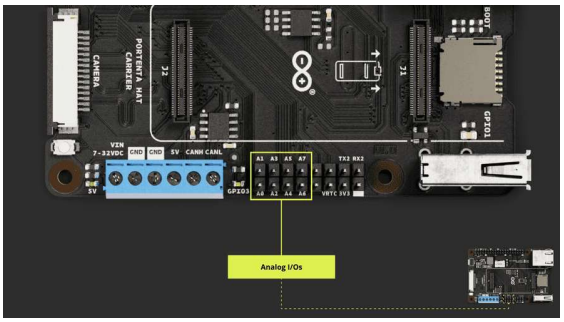
The following example demonstrates how to use the USB interface of the Portenta Hat Carrier with the Portenta C33 to mount a Mass Storage Device (MSD).

Through this code, users will be able to effectively connect to, read from, and write to a USB storage device, making it easier to interact with external storage via the USB interface.

```
#include <vector>
#include <string>
#include "UsbHostMsd.h"
#include "FATFileSystem.h"

#define TEST_FS_NAME "usb"
#define TEST_FOLDER_NAME "TEST_FOLDER"
#define TEST_FILE "test.txt"
#define DELETE_FILE_DIMENSION 150


USBHostMSD block_device;
FATFileSystem fs(TEST_FS_NAME);

std::string root_folder       = std::string(
std::string folder_test_name  = root_folder
std::string file_test_name    = folder_test_

/* this callback will be called when a Mass
void device_attached_callback(void) {
  Serial.println();
  Serial.println("++++ Mass Storage Device d
  Serial.println();
}

void setup() {
  /*
   *  SERIAL INITIALIZATION
```

Analog Pins

The 16-pin header connector of the Portenta Hat Carrier integrates the analog channels. The analog `A0`, `A1`, `A2`, `A3`, `A4`, `A5`, `A6`, and `A7` are accessible through these pins.



Portenta Hat Carrier Analog Pins

Pin number	Silkscreen	Portenta HD Standard Pin	High-Density Pin
1	A0	ANALOG_A0	J2-73
2	A1	ANALOG_A1	J2-75
3	A2	ANALOG_A2	J2-77
4	A3	ANALOG_A3	J2-79
5	A4	ANALOG_A4	J2-74
6	A5	ANALOG_A5	J2-76
7	A6	ANALOG_A6	J2-78
8	A7	ANALOG_A7	J2-80

The built-in features of the Arduino programming language (`analogRead()` function) can be used to access the eight analog input pins on the Arduino IDE.

Please, refer to the board pinout section of the user manual to find the analog pins on the board.

Using Linux

Using the Portenta X8, you can obtain a voltage reading that falls within a *0 - 65535* range. This reading corresponds to a voltage between 0 and 3.3 V. To fetch this reading, use the command:

```
cat /sys/bus/iio/devices/iio\:device0/in_voltage
```

Where `<adc_pin>` is the number of the analog pin to read. For example, in the case of `A0`:

```
cat /sys/bus/iio/devices/iio\:device0/in_voltage
```

If you are working in Python®, the command can be implemented as shown in the script below:

```
def read_adc_value(adc_pin):
  try:
    with open(f'/sys/bus/iio/devices/iio:device
        return int(file.read().strip())
  except FileNotFoundError:
    print(f"ADC pin {adc_pin} not found!")
    return None

if __name__ == "__main__":
  adc_pin = input("Enter ADC pin number: ")
  value = read_adc_value(adc_pin)

  if value is not None:
    print(f"Value from ADC pin {adc_pin}: {valu

    # Mapping between 0-3.3 V
    new_value = (float) (value/65535)*3.3
    print(f"Value mapped between 0-3.3 V: {new_
```

Using Arduino IDE

The following example snippet, compatible with Portenta H7, shows how to read the voltage value from a potentiometer on `A0`. It will then display the readings on the Arduino IDE Serial Monitor.

```
// Define the potentiometer pin and variable to
int potentiometerPin = A0;
int potentiometerValue = 0;

void setup() {
  // Initialize Serial communication
  Serial.begin(9600);
}

void loop() {
  // Read the voltage value from the potentiome
  potentiometerValue = analogRead(potentiometer

  // Print the potentiometer voltage value to t
  Serial.print("- Potentiometer voltage value:
  Serial.println(potentiometerValue);

  // Wait for 1000 milliseconds
  delay(1000);
}
```

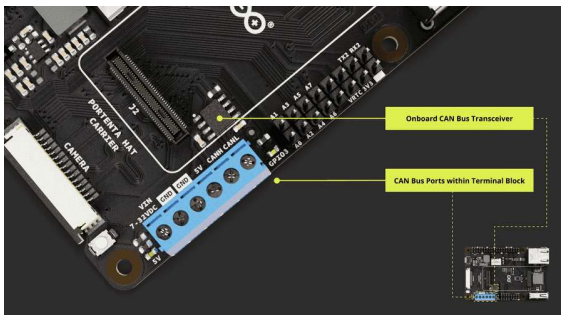
The following example can be considered for Portenta C33:

```
#include "analogWave.h" // Include the library

analogWave wave(DAC);   // Create an instance o

int freq = 10;  // in hertz, change accordingly

void setup() {
  Serial.begin(115200);  // Initialize serial c
  wave.sine(freq);       // Generate a sine wav
}

void loop() {
  // Read an analog value from pin XX and map i
  freq = map(analogRead(XX), 0, 1024, 0, 10000)

  // Print the updated frequency to the serial
  Serial.println("Frequency is now " + String(f

  wave.freq(freq);  // Set the frequency of the
  delay(1000);      // Delay for one second bef
}
```

CAN FD (Onboard Transceiver)

The Portenta Hat Carrier features a dedicated CAN bus connected to a screw terminal block. It uses the *TJA1049* module, a high-speed CAN FD transceiver integrated within the Portenta Hat Carrier.

The TJA1049 module supports ISO 11898-2:2016, SAE J2284-1, and SAE J2284-5 standards over the CAN physical layer, guaranteeing stable communication during the CAN FD fast phase.



Portenta Hat Carrier CAN Interface

Since CAN FD is part of the screw terminal block, we have highlighted the CAN bus ports within the screw terminal block pinout for reference.

Pin number	Silkscreen	Power Net	Portenta HD Standard Pin	High-Density Pin	Interface
1	VIN 7-32VDC	INPUT_7V-32V			
2	GND	GND	GND	J1-22, J1-31, J1-42, J1-47, J1-54, J2-24, J2-33, J2-44, J2-57, J2-70	
3	GND	GND	GND	J1-22, J1-31, J1-42, J1-47, J1-54, J2-24, J2-33, J2-44, J2-57, J2-70	
4	5V	+5V	VIN	J1-21, J1-24, J1-32, J1-41, J1-48	
5	**CANH**			**J1-49 (Through U1)**	**CAN BUS - CANH**
6	**CANL**			**J1-51 (Through U1)**	**CAN BUS - CANL**

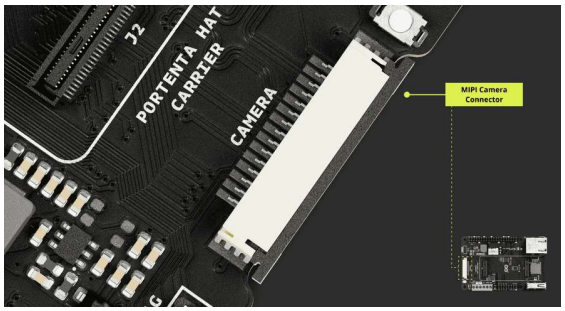
For stable CAN bus communication, it is recommended to install a 120 Ω termination resistor between CANH and CANL lines.

More information on how to use the CAN Bus protocol can be found within CAN Bus section under Pins chapter.

MIPI Camera

The Portenta X8 can interact with MIPI cameras through the dedicated camera connector. As a quick note, the out-of-the-box Alpine shell does not support certain commands directly through the ADB shell.

On the other hand, the Portenta H7 and C33 have no MIPI interface, so they cannot use the camera connector.



Portenta Hat Carrier MIPI Camera

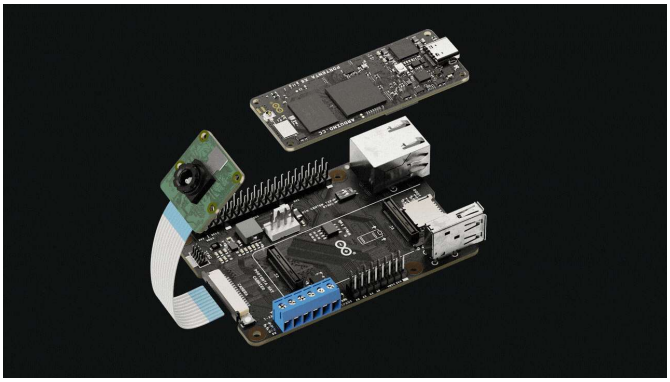
The MIPI connector is distributed as follows:

Pin number	Power Net	Portenta HD Standard Pin	High-Density Pin	Interface
1	GND	GND	J1-22, J1-31, J1-42, J1-47, J1-54	
2		CAM_D0_D0_N	J2-16, J2-24, J2-33, J2-44, J2-57, J2-70	
3		CAM_D1_D0_P	J2-14	
4	GND	GND	J1-22, J1-31, J1-42, J1-47, J1-54, J2-24, J2-33, J2-44, J2-57, J2-70	
5		CAM_D2_D1_N	J2-12	
6		CAM_D3_D1_P	J2-10	
7	GND	GND	J1-22, J1-31, J1-42, J1-47, J1-54, J2-24, J2-33, J2-44, J2-57, J2-70	
8		CAM_CK_CK_N	J2-20	
9		CAM_VS_CK_P	J2-18	
10	GND	GND	J1-22, J1-31, J1-42, J1-47, J1-54, J2-24, J2-33, J2-44, J2-57,	

Pin number	Power Net	Portenta HD Standard Pin	High-Density Pin	Interface
11		GPIO_5	J2-56	
12		NC	NC	
13		I2C1_SCL	J1-45	I2C 1 SCL
14		I2C1_SDA	J1-43	I2C 1 SDA
15	+3V3_PORTENTA	VCC	J2-23, J2-34, J2-43, J2-69	

As mentioned before, the Portenta Hat Carrier supports the MIPI camera if paired with the Portenta X8. The flex cable can be used to interface a compatible camera with the platform. Compatible camera devices are as follows:

- OmniVision OV5647 sensor (Raspberry Pi® Camera Module 1)
- Sony IMX219 sensor (Raspberry Pi® Camera Module 2)



Using Linux

The following commands, using the Portenta X8 environment, allow you to capture a single frame and stream video at 30 FPS (Frames per Second) for 10 seconds from the Raspberry Pi Camera v1.3, which is based on the **OV5647 CMOS** sensor.

First, we need to set environment variables and specify the overlays for our camera and board setup:

```
fw_setenv carrier_custom 1
fw_setenv overlays ov_som_lbee5kl1dx ov_som_x8h7
```

The U-Boot environment variables are modified with the above commands and `fw_setenv` sets the changes which are already persistent. The following command sequences can be used on

```
setenv carrier_custom 1
setenv overlays ov_som_lbee5kl1dx ov_som_x8h7 ov
saveenv
```

Define the runtime directory for `Wayland` and load the necessary module for the OV5647 camera:

```
export XDG_RUNTIME_DIR=/run # location of waylar
modprobe ov5647_mipi
```

Before capturing or streaming, we need to check the supported formats and controls of the connected video device:

```
v4l2-ctl --list-formats-ext --device /dev/video0
v4l2-ctl -d /dev/video0 --list-ctrls
```

Using `GStreamer`, capture a single frame in `JPEG` format:

```
export GST_DEBUG=3
gst-top-1.0 gst-launch-1.0 -v v4l2src device=/de
```

This command allows the user to capture one frame and save it as `/tmp/test.jpg`. The following command is used to stream video at 30FPS for approximately 10 seconds using `GStreamer`:

```
gst-top-1.0 gst-launch-1.0 -v v4l2src device=/de
```

This command allows the user to capture 300 frames at 30 FPS, which equals 10 seconds of video, and displays them using the `waylandsink`.

Following these steps, you will be able to successfully capture and stream video from the Raspberry Pi Camera v1.3 based on the OV5647 sensor.

For enhanced image quality, we recommend using a MIPI camera module with an integrated Image Signal Processor (ISP).

PWM Fan Control

The Portenta Hat Carrier is designed to be a thermal dissipation reference carrier for Portenta X8, including dedicated Pulse Width Modulation (PWM) pins for external fan control. The principle of PWM involves varying the width of the pulses sent to the device, in this case, a fan, to control its speed or position.

The fan can be connected via PWM pins available on the Portenta Hat Carrier. The connector has the following structure:

Pin number	Silkscreen	Power Net	Portenta HD Standard Pin	High-Density Pin
1	PWM9		PWM_9	J2-68
2	N/A			
3	5V	+5V	VIN	J1-21, J1-24, J1-32, J1-41, J1-48
4	GND	GND	GND	J1-22, J1-31, J1-42, J1-47, J1-54, J2-24, J2-33, J2-44, J2-57, J2-70

Using Linux

The fan's speed can be controlled using the following code sequence when you are using the Portenta X8 within the Linux environment.

Export the PWM channel:

```
1  echo 9 > /sys/class/pwm/pwmchip0/export
```

Set the PWM period. By defining the period, you determine the duration of one PWM "cycle". Here, we set it to 100,000, representing 100,000 nanoseconds or 100 microseconds:

```
echo 100000 > /sys/class/pwm/pwmchip0/pwm9/perio
```

The following command sets the "ON" duration within the given period. A 50% duty cycle, for instance, means the signal is on for half the period and off for the other half:

```
echo 50000 > /sys/class/pwm/pwmchip0/pwm9/duty_c
```

We will then enable the PWM channel exported previously:

```
echo 1 > /sys/class/pwm/pwmchip0/pwm9/enable
```

You can use the following command if you want to monitor the temperature of the device or environment (optional step):

```
cat /sys/devices/virtual/thermal/thermal_zone0/t
```

It can be translated into a Python® script to automate the command sequence:

```
def setup_pwm(pwm_chip, pwm_channel, period, 
  base_path = f"/sys/class/pwm/pwmchip{pwm_ch

  # Export the PWM channel
  with open(f"{base_path}/export", "w") as f:
      f.write(str(pwm_channel))

  # Set period
  with open(f"{base_path}/pwm{pwm_channel}/pe
      f.write(str(period))

  # Set duty cycle
  with open(f"{base_path}/pwm{pwm_channel}/du
      f.write(str(duty_cycle))

  # Enable the PWM channel
  with open(f"{base_path}/pwm{pwm_channel}/en
      f.write("1")

def get_thermal_temperature(zone=0):
  with open(f"/sys/devices/virtual/thermal/th
    return int(f.read().strip())


if __name__ == "__main__":
  # Set up PWM
  setup_pwm(0, 9, 100000, 50000)  # 50% duty

```

If you are logged in with normal privileges, the speed of the fan can be controlled using the following instruction sequence. Export the PWM channel using the command below:

```
echo 9 | sudo tee /sys/class/pwm/pwmchip0/export
```

Set the PWM period:

```
echo 100000 | sudo tee /sys/class/pwm/pwmchip0/p
```

Determine the duty cycle at 50%:

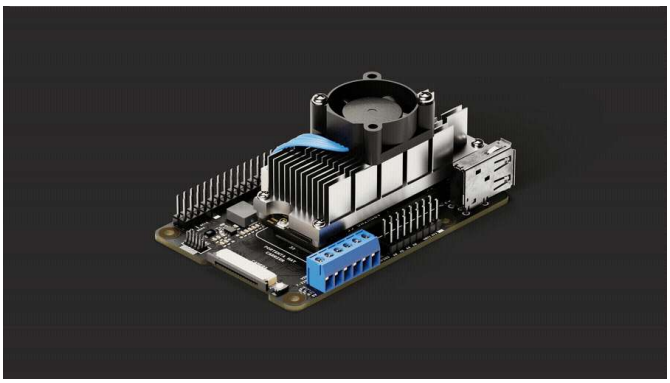
```
echo 50000 | sudo tee /sys/class/pwm/pwmchip0/pw
```

And activate the PWM channel:

```
echo 1 | sudo tee /sys/class/pwm/pwmchip0/pwm9/e
```

Consider the following Python® script if you would like to automate the command sequence:

```
import subprocess

def setup_pwm(pwm_chip, pwm_channel, period, du
  base_path = f"/sys/class/pwm/pwmchip{pwm_chip

  # Export the PWM channel
  subprocess.run(f"echo {pwm_channel} | sudo te

  # Set period
  subprocess.run(f"echo {period} | sudo tee {ba

  # Set duty cycle
  subprocess.run(f"echo {duty_cycle} | sudo tee

  # Enable the PWM channel
  subprocess.run(f"echo 1 | sudo tee {base_path

if __name__ == "__main__":
  setup_pwm(0, 9, 100000, 50000)  # 50% duty
```



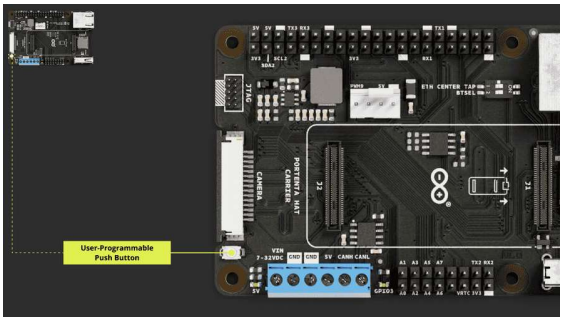
By understanding the fundamentals of PWM and leveraging the capabilities of the Portenta Hat Carrier, you can effectively regulate fan speed as part of the main feature, ensuring optimal cooling performance and longevity of the device of interest.

Storage and Boot Options

Storage and boot-related options are provided to manage the device's data storage and control its operational sequences. Dive into this sub-section to understand the onboard storage options and boot initialization mechanisms with user-programmable actuators.

The Portenta Hat Carrier boasts a streamlined, user-centric design with its multifunctional push button. The button is designed for general user-programmable functions.

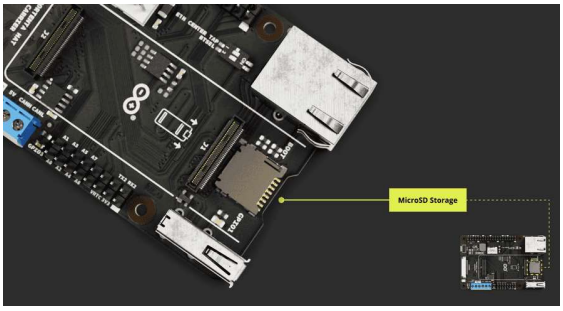
A single button press can be customized according to the application's needs. Whether you need to start a specific event, switch between various states, or execute a particular action, this button is equipped for diverse implementations.



Portenta Hat Carrier Onboard Buttons

MicroSD Storage

The available microSD card slot offers the advantage of expanded storage. This is especially beneficial for processing large volumes of log data, whether from sensors or the onboard computer registry.



Portenta Hat Carrier microSD Expansion Slot

The following table shows an in-depth connector designation:

Pin number	Silkscreen	Power Net	Portenta HD Standard Pin	High-Density Pin
1	N/A		SDC_D2	J1-63
2	N/A		SDC_D3	J1-65
3	N/A		SDC_CMD	J1-57
4	N/A	VDD_SDCARD	VSD	J1-72
5	N/A		SDC_CLK	J1-55

Pin number	Silkscreen	Power Net	Portenta HD Standard Pin	High-Density Pin
				J2-24, J2-33, J2-44, J2-57, J2-70
7	N/A		SDC_D0	J1-59
8	N/A		SDC_D1	J1-61
CD1	N/A		SDC_CD	J1-67
CD2	N/A	GND	GND	J1-22, J1-31, J1-42, J1-47, J1-54, J2-24, J2-33, J2-44, J2-57, J2-70

Using Linux

To begin using a microSD card with Portenta X8, please use the following command to pull a Docker container that assists in setting up the necessary elements for interacting with the microSD card:

```
1 docker run -it --cap-add SYS_ADMIN --device /dev
```

The command above will run the image immediately after the container image has been successfully pulled. You will find yourself inside the container once it is ready for use.

You will need to identify the partition scheme where the microSD card is located. If a partition table does not exist for the microSD card, you will have to use the `fdisk` command to create its partitions.

Inside the container, you can use the following commands.

To determine if the Portenta X8 has recognized the microSD card, you can use one of the following commands:

```
1 lsblk
2
3 # or
4 fdisk -l
```

The microSD card usually appears as `/dev/mmcblk0` or `/dev/sdX`. Where X can be a, b, c, etc. depending on other connected storage devices.

Before accessing the contents of the microSD card, it needs to be mounted. For convenient operation, create a directory that will serve as the mount point:

```
mkdir -p /tmp/sdcard
```

Use the following command to mount the microSD card to the previously created directory. Ensure you replace `XX` with the appropriate partition number (e.g., p1 for the first partition):

```
mount /dev/mmcblk1p1 /tmp/sdcard
```

Navigate to the mount point and list the contents of the SD card:

```
cd /tmp/sdcard
ls
```

To write data to the microSD card, you can use the `echo` command. For example, type the following code to create a file named `hello.txt` with the content `"Hello World Carrier!"`:

```
echo "Hello World Carrier!" > hello.txt
```

To read the contents of the file you have just created:

```
cat hello.txt
```

This will print on your shell the contents that were saved to the `hello.txt` file.

Once you are done with the operations related to the microSD card, it is important to unmount it properly:

```
umount /tmp/sdcard
```

If you need to format the SD card to the *ext4* filesystem, use the

```
mkfs.ext4 /dev/mmcblk1p1
```

Using Arduino IDE

To learn how to use the microSD card slot for enhanced storage with the Arduino IDE, please follow this guide.

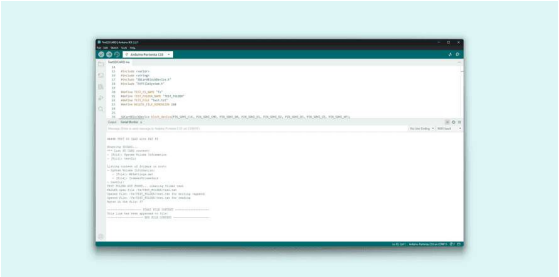
For Portenta H7, you can use the following Arduino IDE script to test the mounted SD card within the Portenta Hat Carrier:

```
#include "SDMMCBlockDevice.h"
#include "FATFileSystem.h"

SDMMCBlockDevice block_device;
mbed::FATFileSystem fs("fs");

void setup() {
  Serial.begin(9600);
  while (!Serial);

  Serial.println("Mounting SDCARD...");
  int err =  fs.mount(&block_device);
  if (err) {
    // Reformat if we can't mount the filesys
    // this should only happen on the first b
    Serial.println("No filesystem found, form
    err = fs.reformat(&block_device);
  }
  if (err) {
     Serial.println("Error formatting SDCARD
     while(1);
  }

  DIR *dir;
  struct dirent *ent;
  int dirIndex = 0;

  Serial.println("List SDCARD content: ");
```

For Portenta C33, consider the following script for testing a mounted SD card.

```
#include <vector>
#include <string>
#include "SDCardBlockDevice.h"
#include "FATFileSystem.h"

#define TEST_FS_NAME "fs"
#define TEST_FOLDER_NAME "TEST_FOLDER"
#define TEST_FILE "test.txt"
#define DELETE_FILE_DIMENSION 150


SDCardBlockDevice block_device(PIN_SDHI_CLK,
FATFileSystem fs(TEST_FS_NAME);

std::string root_folder       = std::string(
std::string folder_test_name  = root_folder
std::string file_test_name    = folder_test_

void setup() {
  /*
   *  SERIAL INITIALIZATION
   */
  Serial.begin(9600);
  while(!Serial) {

  }

  /* list to store all directory in the root
```

Once the script has successfully compiled, the result should resemble the following image:



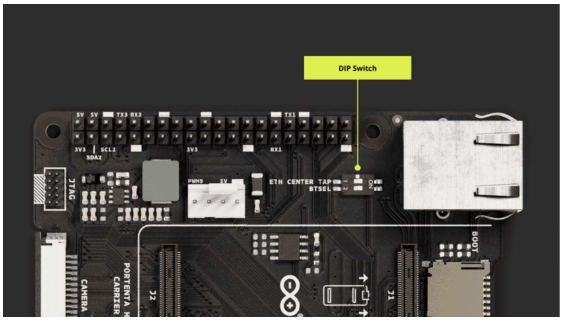
Portenta Hat Carrier with Portenta C33 SD Card Test

Configuration and Control

Configuration and control features allow the user to customize the device's behavior for their specific needs. If you are interested in learning how to set up network connectivity or adjust switch configurations, follow the section below.

DIP Switch Configuration

The Portenta Hat Carrier incorporates a DIP switch, giving users the ability to manage the behavior of the board. The



Portenta Hat Carrier DIP switch

For configurations when the Portenta Hat Carrier is combined with the Portenta X8, the DIP switch governs these settings:

DIP Switch Designation	Position: ON	Position: OFF
ETH CENTER TAP	Ethernet Disabled	Ethernet Enabled
BTSEL	Flashing Mode (ON)	-

Setting the *BTSEL* switch to the `ON` position will place the board in *Flashing Mode*, allowing to update the OS Image of the Portenta X8.

When the Portenta Hat Carrier is combined with either the Portenta H7 or C33, the DIP switch adjustments are as follows:

DIP Switch Designation	Position: ON	Position: OFF
ETH CENTER TAP	Ethernet Enabled	Ethernet Disabled
BTSEL	Not used	Not used

This flexibility ensures that the Portenta Hat Carrier remains adaptable to the unique needs of each paired Portenta board.

Network Connectivity

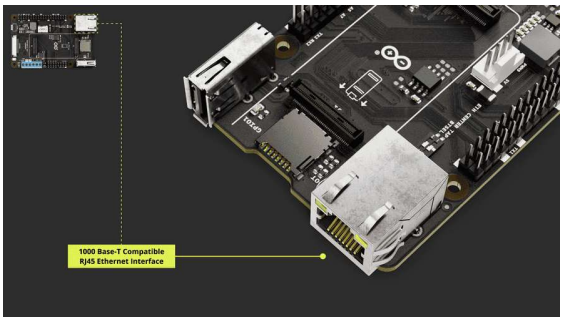
The Portenta Hat Carrier significantly augments the networking functionalities of the devices within the Portenta family through its integrated Ethernet port. Additionally, the Portenta devices destined for pairing with this carrier are inherently equipped with Wi-Fi® and Bluetooth® capabilities.

Thus, when conceptualizing and executing project developments, the user can proficiently exploit both the wired and wireless communication capabilities. The inherent wireless attributes of the Portenta devices, combined with the carrier's sophisticated onboard components and adaptable protocol choices, enable a comprehensive suite of communication solutions ideal for a wide range of applications.

Ethernet

The Portenta HAT Carrier features a gigabit Ethernet port with an RJ45 connector model *TRJG16414AENL* with integrated magnetics. These magnetics are crucial for voltage isolation, noise suppression, signal quality maintenance, and rejecting common mode noise, ensuring adherence to waveform standards.

The connector supports the *1000BASE-T* standard, complying with *IEEE 802.3ab*, guaranteeing high-speed, reliable network connections for data-intensive industrial applications.



Portenta Hat Carrier Ethernet Port

The following table shows an in-depth connector designation:

Pin number	Silkscreen	Power Net	Portenta HD Standard Pin	High-Density Pin
1	N/A	GND	GND	J1-22, J1-31, J1-42, J1-47, J1-54, J2-24, J2-33, J2-44, J2-57, J2-70
2	ETH CENTER TAP			
3	N/A		ETH_D_P	J1-13
4	N/A		ETH_D_N	J1-15
5	N/A		ETH_C_P	J1-9
6	N/A		ETH_C_N	J1-11
7	N/A		ETH_B_P	J1-5
8	N/A		ETH_B_N	J1-7
9	N/A		ETH_A_P	J1-1
10	N/A		ETH_A_N	J1-3
11	N/A		ETH_LED2	J1-19
12	N/A	GND	GND	J1-22, J1-31, J1-42, J1-47, J1-54, J2-24, J2-33, J2-44, J2-57, J2-70
13	N/A		N/A	

Ethernet connection speeds differ based on the associated Portenta board:

- With the Portenta X8: The system supports 1 Gbit Ethernet.
- When combined with the Portenta H7 or C33: The performance is limited to 100 Mbit Ethernet.

To configure the Ethernet settings, depending on the paired Portenta board, one must use the provided DIP switch on the Portenta Hat Carrier. The following table shows the specific DIP switch configuration needed to enable Ethernet on the carrier:

Mounted Portenta Device	ETH CENTER TAP DIP SWITCH
Portenta X8	Position: OFF
Portenta H7/C33	Position: ON

For an in-depth understanding of the DIP switch, kindly refer to this section.

It is advisable to connect the Portenta X8 through the Portenta HAT Carrier to a device with DHCP server capabilities, such as a network router, to ease the automatic assignment of an IP address. DHCP will allow the Portenta X8 to communicate with other devices on the network without manual IP configuration. Employing DHCP simplifies device management, supports dynamic reconfiguration, and provides an advantage for applications involving many devices.

In case you want to assign a manual IP to your device, or even create a direct network between your computer and your board, you can follow the multiple procedures available depending on your network devices and operating system.

Ethernet Interface With Linux

Using the Portenta X8 in combination with the Hat Carrier allows you to evaluate the Ethernet speed between your device and your computer in your network. First, ensure that the Portenta X8 is mounted on the Hat Carrier, and then connect them using an RJ45 LAN cable to your local network. Be sure that your computer and your devices are connected to the same network and are on the same IP range, been capable of seeing each other.

Subsequently, open a terminal to access the shell of the Portenta X8 with admin (root) privileges.

```
adb shell
sudo su -
```

When prompted, enter the password `fio`. To measure the bandwidth, we will use the *iperf3* tool, which is available **here**.

To use the *iperf3* tool, we will set the Portenta X8 and Hat Carrier as the Server and the controlling computer as the Client. The commands will measure the bandwidth between the Portenta Hat Carrier with Portenta X8 and the computer. For a deeper understanding of *iperf3*, refer to its **official documentation**.

Begin by setting up the Portenta Hat Carrier with Portenta X8 as the Server. For the configuration of the necessary files to establish *iperf3* on the device, follow the steps for *Linux and Cygwin* under *General Build Instructions* available **here**. In this case, we need *aarch64 / arm64* version, thus we need to execute the following commands:

```
mkdir -p ~/bin && source ~/.profile
```

```
wget -qO ~/bin/iperf3 https://github.com/userdoc
```

```
chmod 700 ~/bin/iperf3
```

Once installed, *iperf3* will be ready on your device. To ensure it operates without issues, run:

```
chmod +x iperf3
```

By following the provided instructions, the tool should be located in the Linux shell at:

```
# ~bin/
```

Check the tool's version using the following command:

```
./iperf3 -v
```

Activate the Portenta Hat Carrier with Portenta X8 as a Server using the command:

```
./iperf3 -s
```

This will set the Server to wait for connections via its IP address. It listens on port `5201` by default. To use an alternative port, append `-p` and your chosen port number:

```
./iperf3 -s -p <Port Number>
```

To identify the IP address of the Portenta X8, you can use either of the following commands and search for `eth0` which provides the network information related to the Ethernet connection:

```
ifconfig

# Or
ip addr show
```

Next, set up your computer as a Client. In this shared **repository**, select and download a version suitable for your system, like Windows x64.

Once you extract the content, you will notice the *iperf3* file structure as follows:

```
iperf3
    |___bin
    |___include
    |___lib
    |___share
```

Navigate to *bin* and launch a terminal to prepare to use the tool. You can now begin a simple speed test using the following command.

```
# For Linux shell
iperf3 -c <Server IP>

# For Windows
.\iperf3.exe -c <Server IP>
```

This will set the computer as a Client and connect to the configured IP address. If a specific Port needs to be assigned, the following command will allow you to make such a configuration:

```
.\iperf3.exe -c <Server IP> -p <Port Number>
```

Upon starting the test, you will see the amount of data transferred and the corresponding bandwidth rate. With this, you will be able to verify the Ethernet connection and its performance.

Going forward, we can use the following examples to test out Ethernet connectivity.

If you desire to use Portenta X8 paired with Portenta Hat Carrier, please consider following Python® scripts. These scripts use the `socket` library used to create the socket and establish a computer network.

The below script would be used for **Server side (TCP/IP)** operations:

```
#!/usr/bin/env python3

import socket

def start_server():
  HOST = '127.0.0.1'  # Localhost
  PORT = 65432        # Port to listen on

  with socket.socket(socket.AF_INET, socket.SOC
    s.bind((HOST, PORT))
    s.listen()
    print('Server is listening...')
    conn, addr = s.accept()
    with conn:
      print('Connected by', addr)
      while True:
        data = conn.recv(1024)
        if not data:
            break
        conn.sendall(data)

if __name__ == "__main__":
    start_server()
```

The Server-side script is set to wait for incoming connections on `127.0.0.1` (localhost) at port `65432`. These two properties can be modified later at your preference. When a Client connects, the server waits for incoming data and simply sends back whatever it receives, behaving as an echo server.

```
#!/usr/bin/env python3

import socket

def start_client():
  HOST = '127.0.0.1'  # The server's hostname c
  PORT = 65432        # The port used by the se

  with socket.socket(socket.AF_INET, socket.SOC
      s.connect((HOST, PORT))
      s.sendall(b'Hello, server!')
      data = s.recv(1024)

  print('Received', repr(data))

if __name__ == "__main__":
    start_client()
```

The Client-side script connects to the server specified by the `HOST` and `PORT`. These are properties that you change to your preferences. Once connected, it sends a message `"Hello, server!"` and waits for a response.

If you would like to have a single script running both instances, the following script can perform the task using Python®'s built-in `threading` component.

```
import socket
import threading

def server_function():
  HOST = '127.0.0.1'
  PORT = 65432

  with socket.socket(socket.AF_INET, socket.S
    s.bind((HOST, PORT))
    s.listen()
    print('Server is listening...')
    conn, addr = s.accept()
    with conn:
      print('Connected by', addr)
      data = conn.recv(1024)
      if data:
        print('Server received:', repr(data))
        conn.sendall(data)

def client_function():
  HOST = '127.0.0.1'
  PORT = 65432

  with socket.socket(socket.AF_INET, socket.S
    s.connect((HOST, PORT))
    s.sendall(b'Hello, server!')
    data = s.recv(1024)
    print('Client received:', repr(data))
```

The script makes the server start in a separate thread, adding a brief pause using `threading.Event().wait(1)` to confirm it successfully started. It ensures the server is ready to accept connections before the client attempts to connect and send any data.

The client runs on the main thread. Using `server_thread.join()`, the main script waits for the server thread to finish its tasks before exiting.

Ethernet Interface With Arduino IDE

Below is a 'WebClient' example that can be used to test Ethernet connectivity with Portenta H7.

```
#include <PortentaEthernet.h>
#include <Ethernet.h>
#include <SPI.h>

// Enter a MAC address for your controller b
// Newer Ethernet shields have a MAC address
// byte mac[] = { 0xDE, 0xAD, 0xBE, 0xEF, 0xI

// if you don't want to use DNS (and reduce
// use the numeric IP instead of the name fo
//IPAddress server(74,125,232,128);  // nume
char server[] = "www.google.com"; // name ad

// Set the static IP address to use if the DI
IPAddress ip(192, 168, 2, 177);
IPAddress myDns(192, 168, 2, 1);

// Initialize the Ethernet client library
// with the IP address and port of the serve
// that you want to connect to (port 80 is d
EthernetClient client;

// Variables to measure the speed
unsigned long beginMicros, endMicros;
unsigned long byteCount = 0;
bool printWebData = true; // set to false fo

void setup()
```

The following `Web Client` example can be considered for Portenta C33:

```ino
#include <EthernetC33.h>

// if you don't want to use DNS (and reduce y
// use the numeric IP instead of the name for
//IPAddress server(74,125,232,128);  // numer
char server[] = "www.google.com";    // name

// Set the static IP address to use if the DH
IPAddress ip(10, 130, 22, 84);

// Initialize the Ethernet client library
// with the IP address and port of the server
// that you want to connect to (port 80 is de
EthernetClient client;

void setup() {
  // Open serial communications and wait for
  Serial.begin(115200);

  while (!Serial) {
    ; // wait for serial port to connect. Nee
  }

  bool use_dns = true;

  // start the Ethernet connection:
  if (Ethernet.begin() == 0) {
    Serial.println("Failed to configure Ether
```

Wi-Fi® & Bluetooth®

The Portenta Hat Carrier is designed to work flawlessly with wireless features. Among its numerous advantages is its capacity to use Wi-Fi® and Bluetooth® technologies present in the Portenta models like X8, H7, or C33. When these wireless options are activated, they can be effectively combined with the intrinsic capabilities and features that the carrier offers. This combination makes this solution more versatile and powerful for many different projects.

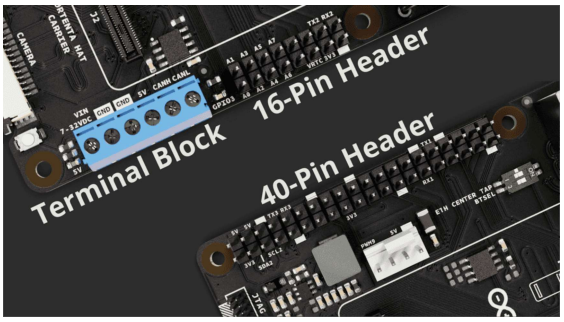
This integration not only broadens the spectrum of use cases for the Portenta Hat Carrier but also ensures that developers can use robust wireless communications in their applications. The effectiveness of onboard capabilities with these wireless features makes the Portenta Hat Carrier an indispensable tool for developers looking for versatile and powerful connectivity solutions.

For a comprehensive understanding of these connectivity options, kindly refer to the specific documentation for each Portenta model.

> Portenta X8 connectivity: **Wi-Fi® configuration** and **Bluetooth®**
>
> Portenta H7 connectivity: **Wi-Fi® access point** and **BLE**

Pins

The Portenta Hat Carrier is a versatile platform, and a significant feature of this carrier is its extensive pin availability. These pins provide a range of functionalities, including power, I/Os, communication, and more.

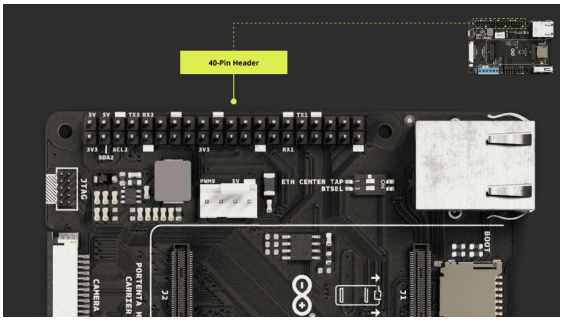


Portenta Hat Carrier Back Side

In this section we will examine the **40 pin** and **16 pin headers** of the Portenta Hat Carrier. These headers are integral to the carrier's interfacing capabilities, providing diverse connectivity options for various applications.

40-Pin Header

The Portenta Hat Carrier provides a 40 pin header that serves as an important interface for numerous applications.



Portenta Hat Carrier 40-Pin Header

To make it easier for developers, here is a comprehensive breakdown of the 40-pin header:

Pin Description	Pin	Pin	Pin Description
VCC (+3V3_PORTENTA)	1	2	VIN (+5V)
I2C2_SDA (I2C 2 SDA)	3	4	VIN (+5V)
I2C2_SCL (I2C 2 SCL)	5	6	GND

Pin Description	Pin	Pin	Pin Description
GND	9	10	SERIAL3_RX (RX3 - UART 3 RX)
GPIO2	11	12	I2S_CK
GPIO6	13	14	GND
SAI_D0	15	16	SAI_CK
VCC (+3V3_PORTENTA)	17	18	SAI_FS
SPI1_MOSI (SPI1 COPI)	19	20	GND
SPI1_MISO (SPI1 CIPO)	21	22	PWM1 (PWM_1)
SPI1_CK (SPI1 SCK)	23	24	SPI1_CS (SPI1 CE)
GND	25	26	PWM2 (PWM_2)
I2C0_SDA (I2C 0 SDA)	27	28	I2C0_SCL (I2C 0 SCL)
SERIAL1_RX (RX1- UART 1 RX)	29	30	GND
PWM3 (PWM_3)	31	32	SERIAL1_TX (TX1- UART 1 TX)
PWM4 (PWM_4)	33	34	GND
I2S_WS (I2S WS)	35	36	PWM5 (PWM_5)
PWM6 (PWM_6)	37	38	I2S_SDI (I2S SDI)
GND	39	40	I2S_SDO (I2S SDO)

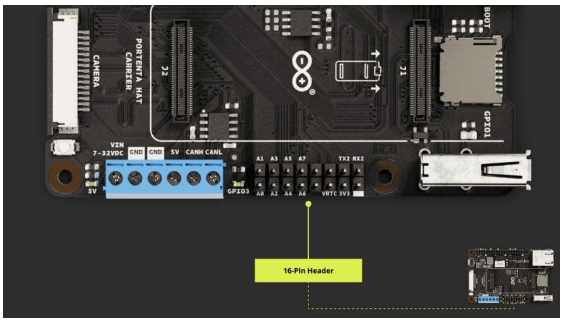
This layout is designed to ensure that developers have a clear understanding of each pin and its function.

16-Pin Header

The Portenta Hat Carrier features a 16-pin male header connector dedicated to analog input but also offers a variety of other functionalities. The table below provides a detailed mapping:

Pin Description	Pins	Pins	Pin Description
ANALOG_A0 (A0)	1	2	ANALOG_A1 (A1)
ANALOG_A2 (A2)	3	4	ANALOG_A3 (A3)
ANALOG_A4 (A4)	5	6	ANALOG_A5 (A5)
ANALOG_A6 (A6)	7	8	ANALOG_A7 (A7)
PWM7 (PWM_7)	9	10	PWM8 (PWM_8)
LICELL (RTC Power Source)	11	12	GPIO0 (PWM4)
VCC (+3V3_PORTENTA)	13	14	SERIAL2_TX (TX2 - UART 2 TX)
GND	15	16	SERIAL2_RX (RX2 - UART 2 RX)

A visual representation of the header can be seen in the image



Portenta Hat Carrier 16-Pin Header

It is characterized as follows:

Analog Pins: It integrates eight dedicated pins for analog channels. It ranges from *A0 ~ A7*, and each of these pins serves a unique analog channel, facilitating a range of analog signal measurements.

PWM Pins: Integrates dedicated PWM pins within the header. Pin 9 is labeled *PWM7 (PWM_7)*, and Pin 10 is identified as *PWM8 (PWM_8)*.

Additionally, Pin 12, although a General-Purpose Input/Output (GPIO0), also supports PWM and is labeled as PWM4.

Serial Pins: It integrates UART 2 functionalities. Pin 14 is the transmit function, identified as *SERIAL2_TX* or *TX2*, while Pin 16 is dedicated to the receive function, labeled as *SERIAL2_RX* or *RX2*.

Power and Grounding: Pin 11, labeled as LICELL, serves as the *Real Time Clock (RTC)* power source.

For providing a voltage source, Pin 13 offers a 3.3 V output, specifically for the Portenta module, and is marked as *VCC (+3V3_PORTENTA)*. The Ground for this header is accessible via Pin 15, designated simply as *GND*.

GPIO Pins

Understanding and managing the General-Purpose Input/Output (GPIO) pins on your device can be crucial for many applications. The following script is designed to display all the GPIOs available on the 40-pin connector of the Portenta Hat Carrier paired with Portenta X8.

Using Linux

Next conditions will help you properly set the hardware to test GPIO controls:

1. Begin by positioning the Portenta-X8 securely onto the Portenta Hat Carrier. Make sure the High-Density connectors are securely connected.
2. Each GPIO on the Portenta Hat Carrier is versatile and robust, designed to safely accept input voltages ranging between 0.0 V and 3.3 V. This input range ensures compatibility with an array of sensors and external devices.
3. To prevent floating states and offer a consistent and predictable behavior, internal pull-ups are automatically enabled on all input pins. This default configuration means that, unless actively driven low, the pins will naturally read as high (or 3.3 V).

When all conditions are set and in place, access the Portenta X8's shell with admin (root) access as follows:

```
adb shell
sudo su -
```

Enter the password `fio` when prompted. Next, access the `x8-devel` Docker container with the command:

```
docker exec -it x8-devel sh
```

Navigate to the directory containing the GPIO example, named `gpios.py`:

```
cd root/examples/portenta-hat-carrier
```

Run the `gpios.py` script to read the status of all available GPIOs on the 40-pin header:

```
#!/usr/bin/env python3

# created 12 October 2023
# by Riccardo Mereu & Massimo Pennazio

import os

if os.environ['CARRIER_NAME'] != "rasptenta":
    print("This script requires Portenta HAT ca
    exit(1)

import Portenta.GPIO as GPIO

GPIO.setmode(GPIO.BCM)

all_pins_in_header = [2,3,4,5,6,7,8,9,10,11,12,

GPIO.setup(all_pins_in_header, GPIO.IN)

for pin in all_pins_in_header:
    try:
        print(GPIO.input(pin))
    except RuntimeError as e:
        print("Error reading GPIO %s: %s" % (st

GPIO.cleanup()
```

This script will help you verify the following considerations:

- Avoid manually checking each pin by having a consolidated overview of all GPIOs' statuses.
- By staying within the specified voltage range, you ensure the longevity of your device and prevent potential damages.
- With the default pull-ups, you can be confident in your readings, knowing that unintentional fluctuations are minimized.

By employing this script, not only do you gain a deeper insight into the state of your GPIOs, but you also save valuable time and reduce the margin for error.

Whether you are debugging, prototyping, or setting up a new project, this script is an invaluable tool for all Portenta Hat Carrier users.

You can also retrieve information about the available GPIOs on the Portenta X8's shell using the following command:

```
cat /sys/kernel/debug/gpio
```

If Portenta Hat Carrier is paired with Portenta H7 or Portenta C33, consider using the following example:

```
const int actPin = 2;           // the number of
const int ledPin = <PD_5/30>;   // User programm
int actState = 0;               // variable for

void setup() {
  pinMode(ledPin, OUTPUT);      // initialize th
  pinMode(actPin, INPUT);       // initialize th
}

void loop() {
  actState = digitalRead(actPin);  // read the

  if (actState == HIGH) {          // if the GP
    digitalWrite(ledPin, HIGH);
  } else {
    digitalWrite(ledPin, LOW);
  }
}
```

This example uses a designated GPIO pin to set the user-programmable LED on the Portenta Hat Carrier to either a HIGH or LOW state.

Alternatively, the following example controls the user-programmable LED on the Portenta Hat Carrier based on potentiometer input:

```
const int potPin = A0;          // the number of
const int ledPin = <PD_5/30>;   // User programm

int potValue = 0;               // value read fr
int ledThreshold = 0;           // PWM value for

void setup() {
  pinMode(ledPin, OUTPUT);       // initialize t
                                 // choose eithe
}

void loop() {
  potValue = analogRead(potPin);
  ledThreshold = map(potValue, 0, 1023, 0, 255)

  if (ledThreshold >= 128){
    digitalWrite(ledPin, HIGH);          // set
  } else {
    digitalWrite(ledPin, LOW);           // set
  }
  delay(10);
}
```

PWM Pins

The Portenta Hat Carrier has 10 digital pins with PWM functionality, mapped as follows:

Pin number	Silkscreen	Portenta HD Standard Pin	High-Density Pin
40-Pin Header			
7	PWM0	PWM_0	J2-59
22	PWM1	PWM_1	J2-61
26	PWM2	PWM_2	J2-63
31	PWM3	PWM_3	J2-65
33	PWM4	PWM_4	J2-67
36	PWM5	PWM_5	J2-60
37	PWM6	PWM_6	J2-62
16-Pin Header			
9	PWM7	PWM_7	J2-64
10	PWM8	PWM_8	J2-66
12	PWM4	GPIO_0	J2-46

Please, refer to the board pinout section of the user manual to find them on the board. All these pins must be configured on the corresponding Portenta.

Using Linux

The following Python® script is designed to control the brightness of a device, such as an LED, by varying the duty cycle of a PWM signal in a Linux environment on Portenta X8.

The script sets up the PWM channel, defines its period, and then, within a loop, modulates the brightness by adjusting the duty cycle. Consider the script below as an example:

```python
#!/usr/bin/env python3

import time
import subprocess

# Define the PWM chip, channel, and other par
PWM_CHIP = 0
PWM_CHANNEL = 9  # Replace with the correct cl
BASE_PATH = f"/sys/class/pwm/pwmchip{PWM_CHIP
PERIOD = 1000000  # 1 second in nanoseconds
FADE_DURATION = 0.03  # 30 milliseconds

# Define brightness and fade amount variables
brightness = 0
fadeAmount = 5 * (PERIOD // 255)  # Scale fad

def setup_pwm():
  subprocess.run(f"echo {PWM_CHANNEL} | sudo
  subprocess.run(f"echo {PERIOD} | sudo tee {
  subprocess.run(f"echo 0 | sudo tee {BASE_PA
  subprocess.run(f"echo 1 | sudo tee {BASE_PA

def set_pwm_brightness(brightness_value):
  duty_cycle = brightness_value * (PERIOD // 
  subprocess.run(f"echo {duty_cycle} | sudo t

if __name__ == "__main__":
  setup_pwm()
```

Using Arduino IDE

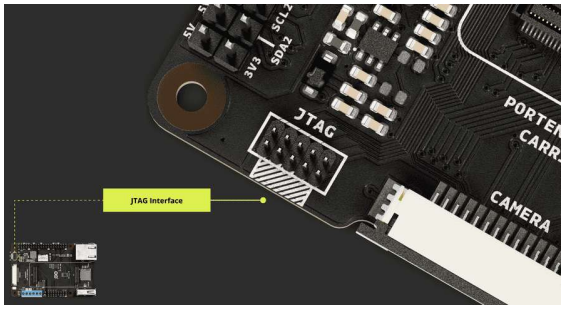
The [`analogWrite()` function included in the Arduino programming language can be used to access the PWM pins.

The example code shown below grabs a pin compatible with PWM functionality to control the brightness of an LED connected to it:

```
const int ledPin = <PWM_X>;  // Use a pin that

void setup() {
  pinMode(ledPin, OUTPUT);  // Configure the pi
}

void loop() {
  // Increase brightness
  for (int brightness = 0; brightness <= 255; b
    analogWrite(ledPin, brightness);
    delay(10);
  }

  // Decrease brightness
  for (int brightness = 255; brightness >= 0; b
    analogWrite(ledPin, brightness);
    delay(10);
  }
```

JTAG Pins

For developers aiming to investigate and understand the intricate details of development, the Portenta Hat Carrier features a built-in JTAG interface. This tool is crucial for hardware debugging, offering real-time observation. Through the JTAG pins, users can smoothly debug and program, guaranteeing accurate and optimal device performance.



Portenta Hat Carrier onboard JTAG pin

The pins used for the JTAG debug port on the Portenta Hat Carrier are the following:

Pin number	Power Net	Portenta HD Standard Pin	High-Density Pin	Interface
1	+3V3_PORTENTA	VCC	J2-23, J2-34, J2-43, J2-69	
2		JTAG_SWD	J1-75	JTAG SWD
3	GND	GND	J1-22, J1-31, J1-42, J1-47, J1-54, J2-24, J2-33, J2-44, J2-57, J2-70	
4		JTAG_SCK	J1-77	JTAG SCK
5	GND	GND	J1-22, J1-31, J1-42, J1-47, J1-54, J2-24, J2-33, J2-44, J2-57, J2-70	
6		JTAG_SWO	J1-79	JTAG SWO
7		NC	NC	
8		JTAG_TDI	J1-78	JTAG TDI
9		JTAG_TRST	J1-80	JTAG TRST

Understanding Device Tree Blobs (DTB) Overlays

Device Tree Blobs (DTB) And DTB Overlays

In the world of embedded systems, *U-boot* and the *Linux kernel* use a concept called **Device Tree Blobs (DTB)** to describe a board's hardware configuration. This approach allows for a unified main source tree to be used across different board configurations, ensuring consistency.

The boards, acting as carriers, allow various peripherals to be connected, such as temperature sensors or accelerometers. These carriers serve as expansion connectors. You might want to connect various peripherals and be able to add or remove them easily.

The concept of modularity is applied to the *DTB*, resulting in **DTB overlays**. The hardware configuration is split into multiple small files, each representing a different peripheral or function in the form of a DTB overlay.

During the early boot stage, these overlays are merged together into a single DTB and loaded into RAM. This approach enables users to select and change configurations with ease. However, it is important to note that changing the hardware configuration requires a system reboot to maintain system stability.

Handling DTB Overlays

To modify and maintain the Device Tree Blob (DTB) overlays of your Portenta X8 so it can support different hardware and devices, please read and execute the following steps.

Custom DTB Overlays

In cases where the required DTB overlay is not readily available and a specific configuration that is not part of the pre-compiled set is needed, it is possible to create customized DTB overlays.

DTB overlays originate from readable source files known as *DTS files*. Users with the respective experience can modify these DTS files and cross-compile them to create tailored overlays suited to their needs.

Automated Load And Carrier Detection

U-boot can be configured to automatically load specific DTB overlays based on the carrier board it detects, in this case the Portenta Hat Carrier, either by probing specific hardware or by

For instance, for a Portenta-X8 placed on a Portenta HAT Carrier, upon logging into the board and executing subsequent commands on the shell, the expected output is as follows:

```
fw_printenv overlays
overlays=ov_som_lbee5kl1dx ov_som_x8h7 ov_carrie

fw_printenv carrier_name
carrier_name=rasptenta

fw_printenv is_on_carrier
is_on_carrier=yes
```

This information is written by U-boot during boot in a step referred to as *auto carrier detection*. You can modify the variables from user space, but after a reboot, they revert to their default state unless the `carrier_custom` variable is set to `1`.

```
fw_setenv carrier_custom 1
```

This serves as an escape mechanism to enable user-based configurations.

```
fw_setenv carrier_custom 1
fw_setenv carrier_name rasptenta
fw_setenv is_on_carrier yes
fw_setenv overlays "ov_som_lbee5kl1dx ov_som_x8h
```

The commands above enable functionalities such as a speed-controlled fan connector, an OV5647 based RPi v1.3 camera, and an IQ Audio Codec Zero audio HAT.

Hardware Configuration Layers

Hardware configuration is divided into the following layers:

- **Layer 0**: System on Module (SoM), prefixed with `ov_som_`.
- **Layer 1**: Carrier boards, prefixed with `ov_carrier_`.
- **Layer 2**: HATs and Cameras, which is usually a concatenation of the carrier name and the hat name or functionality.

EEPROMs, which store identification IDs, are typically defined on *Layer 1* and accessible on *I2C1*. Some HATs may also have EEPROMs according to the Raspberry Pi® standard (*ID_SD*, *ID_SC*),

There are some overlays which add specific functionalities. For example:

- `ov_som_1bee5kl1dx`: Adds Wi-Fi®
- `ov_som_x8h7`: Adds the H7 external microcontroller
- `ov_carrier_rasptenta_base`: Base support for Portenta Hat Carrier

When no known carrier is detected and the Portenta X8 is mounted as the main board, the first two overlays mentioned above are applied by default.

Distinction Between System And Hardware Configuration

The distinction between system and hardware configuration is crucial. System configuration includes settings such as user creation and Wi-Fi® passwords, whereas hardware configuration is explicitly defined through the device tree.

In production environments, the addition of custom compiled device tree overlays is restricted to maintain system integrity and security.

Raspberry Pi® HAT

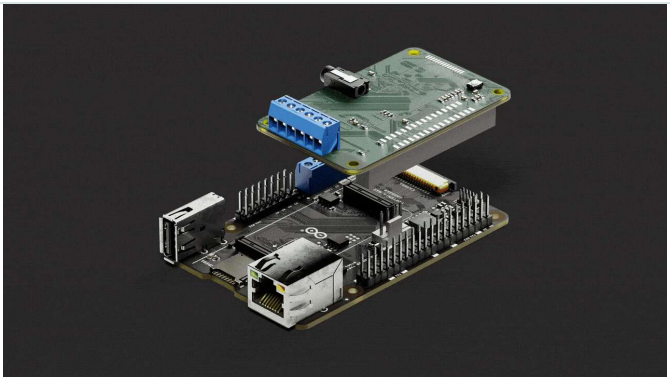
The Portenta Hat Carrier is notable for its compatibility with **Hardware Attached on Top (HAT)** add-on boards.

A **Hardware Attached on Top (HAT)** is known as a standardized add-on module designed to be interfaced with compatible host systems. The HAT concept can be understood as a modular approach to hardware extension.

Following certain design rules to specific mechanical and electronic design criteria, a HAT usually has a built-in memory chip (EEPROM). This allows the host system to automatically recognize and potentially configure itself corresponding to the attached module.

The main objective of a HAT is to augment the functionalities of the host device, allowing for capabilities ranging from sensor integration and display enhancements to advanced audio processing and communication features.

The standardized design of HATs ensures they are compatible and easy to use with various devices. This makes them suitable for both seasoned professionals and enthusiasts.



These is the officially compatible list of HATs:

Stepper Motor HAT: it is a HAT that drives stepper motors, enabling precise control of rotation direction, angle, speed, and steps, suitable for projects like CNC, 3D printers, and robotics. It uses a DRV8825 dual H-bridge motor driver.

RPi Relay Board: it is a HAT that eases the control of high-voltage devices, featuring three channels, and photo coupling isolation. It helps provide safe device switching for various applications.

The example scripts are located within the Docker container. To access these scripts and test them with the Hat mounted, execute the following command:

```
docker exec -it x8-devel sh
```

Upon gaining access to the container, all relevant Portenta Hat Carrier scripts are conveniently located in the `root/examples/portenta-hat-carrier` directory. Alternatively, you also have the option to manually dockerize the scripts to run them within ADB shell of the Portenta X8.

To use the example script, a series of commands can be used. These will help you define and export necessary modules before executing the desired example. To start, the Python® shell can be launched within the container using:

```
python3
```

To use modules like *smbus2*, the following sequence of commands will help you import such modules:

```
import smbus2
```

To use a specific class or function from the module:

```
from smbus2 import SMBus
```

Setting variables is straightforward. Depending on your requirement, assign values in either HEX or DEC:

```
# Value can be in HEX or DEC
variable_name = Value
```

To run an example script within the Python® shell, consider using the following command:

```
with open('example_script.py', 'r') as file:
...     exec(file.read())
```

If you prefer traditional methods of execution, the following command can be used:

```
python3 example_script.py
```

This last command for example is also applicable within ADB shell of the Portenta X8.

The following sections will help you become familiar with the examples found within the `root/examples/portenta-hat-carrier` directory. This directory contains both the *RPi Relay Board* and the *Stepper Motor HAT*. These examples are used within the **Linux** environment.

RPi Relay Board

The *RPi Relay Board* is a dynamic board that consists of three channels: a high-level trigger and two low-level triggers. With the capability to manage up to 250V AC or 30V DC with a current rating of 2A, it operates using the I2C interface, renowned for its high-quality relays.

The continuing script offers an interface for interaction with the Seeed Studio Raspberry Pi Relay Board. Authored by John M. Wargo, it is a refined version of the sample code available on the Seeed Studio Wiki. The script uses the *smbus* library to interface

It can interact with up to four relay ports on the board. Among its various features, it can turn a specific relay on or off, toggle all relays simultaneously, toggle a particular relay's state, and retrieve the status of any relay. Furthermore, it has built-in error handling to ensure that a valid integer relay number is specified.

```
from __future__ import print_function

from smbus2 import SMBus

# The number of relay ports on the relay boa
# This value should never change!
NUM_RELAY_PORTS = 4

# Change the following value if your Relay b
DEVICE_ADDRESS = 0x20  # 7 bit address (will

# Don't change the values, there's no need f
DEVICE_REG_MODE1 = 0x06
DEVICE_REG_DATA = 0xff

bus = SMBus(3)  # 0 = /dev/i2c-0 (port I2C0)


def relay_on(relay_num):
    global DEVICE_ADDRESS
    global DEVICE_REG_DATA
    global DEVICE_REG_MODE1

    if isinstance(relay_num, int):
        # do we have a valid relay number?
        if 0 < relay_num <= NUM_RELAY_PORTS:
            print('Turning relay', relay_num
            DEVICE_REG_DATA &= ~(0x1 << (rel
```

Next script showcases the utility of the relay board functions. At the onset, it activates all the relays and then deactivates them, pausing for a second between these actions.

Subsequently, it sequentially powers each relay on and off, with a one-second intermission in between. In the event of a keyboard interrupt, the script terminates and ensures all the relays are switched off.

```python
#!/usr/bin/python

from __future__ import print_function

import sys
import time

from relay_lib_seeed import *


def process_loop():
    # turn all of the relays on
    relay_all_on()
    # wait a second
    time.sleep(1)
    # turn all of the relays off
    relay_all_off()
    # wait a second
    time.sleep(1)

    # now cycle each relay every second in an
    while True:
        for i in range(1, 5):
            relay_on(i)
            time.sleep(1)
            relay_off(i)
```

For implementation, this script draws functions from `relay_lib_seeed`, which was previously explained. Together, these scripts simplify the control of the RPi Relay Board.

Stepper Motor HAT

Using the capabilities of the Portenta X8 alongside specific modules can increase its performance. One such module is the **drv8825 HAT**, designed specifically for driving stepper motors, especially when paired with the Portenta Hat Carrier.

To use the drv8825 HAT with Portenta Hat Carrier and Portenta X8, please follow these steps:

1. Place the Portenta X8 onto the Portenta Hat Carrier.
2. Align and position the drv8825 HAT on the Portenta Hat Carrier. Make sure to align its 40-Pin header.
3. Proceed by wiring the motor poles. This can be done using either the **A1-A2**, **B1-B2** configurations or the alternative **A3-B3**, **A4-B4** setups. Proper wiring is crucial for achieving the desired rotational motion in the motor.
4. To ensure the system is powered on, connect an external power source using the **VIN-GND** terminals.

5 One distinguishing feature of the drv8825 HAT is the provision for micro-stepping. This feature enhances the precision of motor operations.

Adjust the micro-stepping settings using the DIP switches present on the drv8825 HAT to achieve the desired level of granularity in motor steps.

Once the hardware setup is ready, use the script below to perform a test run of the connected stepper motor:

```
#!/usr/bin/env python3

# created 12 October 2023
# by Riccardo Mereu & Massimo Pennazio

import os

if os.environ['CARRIER_NAME'] != "rasptenta":
    print("This script requires Portenta HAT ca
    exit(1)

from rpi_python_drv8825.stepper import StepperM

motor = StepperMotor(enable_pin, step_pin, dir_

motor.enable(True)        # enables stepper dri
motor.run(6400, True)     # run motor 6400 step
motor.run(6400, False)    # run motor 6400 step
motor.enable(False)       # disable stepper dri
```

The parameters for the stepper motor must be defined. Consider the following parameters as an example:

```
enable_pin = 12
step_pin = 23
dir_pin = 24
mode_pins = (14, 15, 18)
step_type = '1/32'
fullstep_delay = .005
```

For a comprehensive understanding, and perhaps to delve into advanced configurations, Waveshare's Stepper Motor HAT (B) Wiki is an excellent resource. It provides extensive insights and details about the drv8825 HAT.

Communication

The Portenta Hat Carrier extends multiple communication protocols from the Portenta core board. These protocols are

Universal Asynchronous Receiver-Transmitter (UART), JTAG interface, and MIPI tailored for the Portenta X8 camera compatibility. This section offers further details on these protocols.

Dedicated pins are provided as well on the Portenta Hat Carrier for each communication protocol. They are easily accessible via the 40-pin male header connectors, simplifying the process of interfacing with various components, peripherals, and sensors. Additionally, a 16-pin header connector is present to support analog pins, PWM pins, LICELL, and serial communication.

SPI

The Portenta Hat Carrier supports SPI communication via two dedicated ports named `SPI0` and `SPI1`. Both ports are available via High-Density connectors, while `SPI1` is also available over **40-pin connector**. This allows data transmission between the board and other SPI-compatible devices. The pins used in the Portenta Hat Carrier for the SPI communication protocol are the following:

Pin number	Silkscreen	Portenta HD Standard Pin	High-Density Pin	Interface
19	SPI1 COPI	SPI1_MOSI	J2-42	SPI 1 MOSI
21	SPI1 CIPO	SPI1_MISO	J2-40	SPI 1 MISO
23	SPI1 SCK	SPI1_CK	J2-38	SPI 1 CK
24	SPI1 CE	SPI1_CS	J2-36	SPI 1 CS

Please, refer to the **board pinout section** of the user manual to find them on the board.

Using Linux

With admin (root) access, you can use the following commands within the shell for the Portenta X8:

```
sudo modprobe spidev
```

Present sequence of commands is used to enable the SPI device interface on the Portenta X8. After adding the `spidev` module to the system's configuration, the system is rebooted to apply the changes.

```
echo "spidev" | sudo tee > /etc/modules-load.d/s
sudo systemctl reboot
```

Following section configures a service named `my_spi_service` to use the SPI device available at `/dev/spidev0.0`.

```
services:
    my_spi_service:
       devices:
          - '/dev/spidev0.0'
```

Using Arduino IDE

Include the `SPI` **library** at the top of your sketch to use the SPI communication protocol. This can be used with Portenta H7 or C33. The SPI library provides functions for SPI communication:

```
#include <SPI.h>
```

The `setup()` function compiles initial SPI processes, by setting the chip select (`CS) with the proper configuration.

```
void setup() {
  // Set the chip select pin as output
  pinMode(SS, OUTPUT);

  // Pull the CS pin HIGH to unselect the devic
  digitalWrite(SS, HIGH);

  // Initialize the SPI communication
  SPI.begin();
}
```

Following commands can be used to establish transmission and exchange information with an SPI-compatible device.

```
// Replace with the target device's address
byte address = 0x00;

// Replace with the value to send
byte value = 0xFF;

// Pull the CS pin LOW to select the device
digitalWrite(SS, LOW);

// Send the address
SPI.transfer(address);

// Send the value
SPI.transfer(value);

// Pull the CS pin HIGH to unselect the device
digitalWrite(SS, HIGH);
```

I2S

I2S, short for Inter-IC Sound, connects digital audio devices using an electrical serial bus interface.

It operates using three main lines:

SCK (Serial Clock) or BCLK: the clock signal.

WS (Word Select) or FS (Frame Select): Differentiates data for the Right or Left Channel.

SD (Serial Data): transmits the audio data.

The Controller generates the SCK and WS signals, and its frequency is derived from SampleRate x Bits Per Channel x Number of Channels.

In an I2S setup, while one device acts as the Controller, others are in Peripheral mode. Audio data samples can vary from 4 to 32 bits, with higher sample rates and bits offering superior audio quality.

The pins used in the Portenta Hat Carrier for the I2S communication protocol are the following:

Pin number	Silkscreen	Portenta HD Standard Pin	High-Density Pin	Interface
12	I2S CK	I2S_CK	J1-56	I2S CK
35	I2S WS	I2S_WS	J1-58	I2S WS
38	I2S SDI	I2S_SDI	J1-60	I2S SDI
40	I2S SDO	I2S_SDO	J1-62	I2S SDO

SAI - Serial Audio Interface

Serial Audio Interface (SAI) is a versatile protocol for

fixed I2S standard, SAI supports multiple audio data formats and configurations. The carrier works with the following data lines:

D0: This serves as the primary data line, transmitting or receiving audio data.

CK (BCLK): The Bit Clock, governing the rate of individual audio data bit transmission or reception.

FS: Frame Sync, marking the boundary of audio frames, often differentiating channels in stereo audio.

SAI protocol can operate both synchronously and asynchronously, adjusting to various audio system needs. Due to its adaptability, SAI suits complex audio tasks, systems with multi-channel requirements, and specific audio formats.

In essence, SAI offers greater flexibility than I2S, catering to a broader range of audio system configurations.

The pins used in the Portenta Hat Carrier for the SAI protocol are the following:

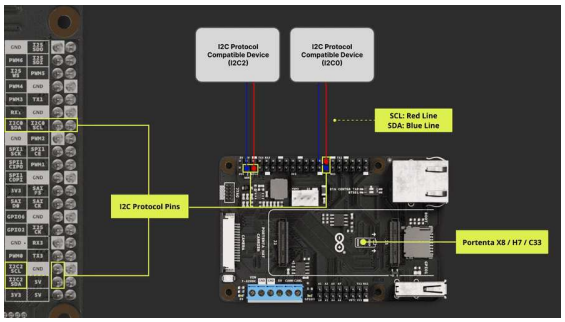
Pin number	Silkscreen	Portenta HD Standard Pin	High-Density Pin	Interface
15	SAI D0	SAI_D0	J2-53	SAI D0
16	SAI CK	SAI_CK	J2-49	SAI CK
18	SAI FS	SAI_FS	J2-51	SAI FS

I2C

The Portenta Hat Carrier supports I2C communication, which allows data transmission between the board and other I2C-compatible devices. The pins used in the Portenta Hat Carrier for the I2C communication protocol are the following:

Pin number	Silkscreen	Portenta HD Standard Pin	High-Density Pin	Interface
27	I2C0 SDA	I2C0_SDA	J1-44	I2C 0 SDA
28	I2C0 SCL	I2C0_SCL	J1-46	I2C 0 SCL
3	I2C2 SDA	I2C2_SDA	J2-45	I2C 2 SDA
5	I2C2 SCL	I2C2_SCL	J2-47	I2C 2 SCL

Please, refer to the pinout section of the user manual to find them on the board. The I^2C pins are also available through the onboard ESLOV connector of the Portenta Hat Carrier.



Portenta Hat Carrier CAN Bus Interface Connection Example

Using Linux

For the Portenta X8, it is possible to use the following commands within the shell when you have admin (root) access:

```
sudo modprobe i2c-dev
```

Present sequence of commands is used to enable the I2C device interface on the Portenta X8. After adding the `i2c-dev` module to the system's configuration, the system is rebooted to apply the changes.

```
echo "i2c-dev" | sudo tee > /etc/modules-load.d/
sudo systemctl reboot
```

Following section configures a service named `my_i2c_service` to use the I2C device available at `/dev/i2c-3`.

```
services:
    my_i2c_service:
       devices:
          - `/dev/i2c-3`
```

Within the Portenta X8 shell, you can use specific commands to quickly test I2C communication with compatible devices. The command below lists all the connected I2C devices:

```
i2cdetect -y <I2C bus>
```

To communicate with and fetch information from a connected I2C device, use:

```
i2cget -y <I2C bus> <device address> <register a
```

Below are some brief examples to help you in using I2C with the Portenta X8 and the Portenta Hat Carrier.

Here, the SMBus (System Management Bus) communication, with SMBus-compatible **libraries**, is established with the device on `/dev/i2c-3`. A byte of data is read from the device at address 80 and offset 0, then printed.

```
from smbus2 import SMBus

# Connect to /dev/i2c-3
bus = SMBus(3)
b = bus.read_byte_data(80, 0)
print(b)
```

The following code initializes the I2C bus using the *smbus2 library* and reads multiple bytes from the device. The `read_i2c_block_data` function reads a block of bytes from the I2C device at a given address.

```
from smbus2 import SMBus

# Initialize the I2C bus
bus = SMBus(3)  # 3 indicates /dev/i2c-3

device_address = 0x1
num_bytes = 2

# Read from the I2C device
data = bus.read_i2c_block_data(device_address,

# Now data is a list of bytes
for byte in data:
    print(byte)
```

The next code shows how to write data to an I2C device using the *smbus2 library*. A byte of data (`value`) is written to a specific address (`device_address`) with a given instruction.

```
from smbus2 import SMBus

# Initialize the I2C bus
bus = SMBus(3)  # 3 indicates /dev/i2c-3

device_address = 0x1
instruction = 0x00
value = 0xFF

# Write to the I2C device
bus.write_byte_data(device_address, instructior
```

In the following code, the **python-periphery** library is used to interact with the I2C device. This is useful if a broad spectrum of protocols are required within the same script. The `I2C.transfer()` method performs both write and read operations on the I2C bus.

A byte is read from the EEPROM at address `0x50` and offset `0x100`, then printed.

```
from periphery import I2C

# Open i2c-0 controller
i2c = I2C("/dev/i2c-3")

# Read byte at address 0x100 of EEPROM at 0x50
msgs = [I2C.Message([0x01, 0x00]), I2C.Message(
i2c.transfer(0x50, msgs)
print("0x100: 0x{:02x}".format(msgs[1].data[0])

i2c.close()
```

Using Arduino IDE

To use I2C communication, include the `Wire` **library** at the top of your sketch. This can be used with Portenta H7 or C33. The `Wire` library provides functions for I2C communication:

```
#include <Wire.h>
```

In the `setup()` function, initialize the I2C library:

```
// Initialize the I2C communication
Wire.begin();
```

Following commands are available to transmit or write with an I2C-compatible device.

```
// Replace with the target device's I2C address
byte deviceAddress = 0x1;

// Replace with the appropriate instruction byt
byte instruction = 0x00;

// Replace with the value to send
byte value = 0xFF;

// Begin transmission to the target device
Wire.beginTransmission(deviceAddress);

// Send the instruction byte
Wire.write(instruction);

// Send the value
Wire.write(value);

// End transmission
Wire.endTransmission();
```

Following commands are available to request or read with an I2C-compatible device.

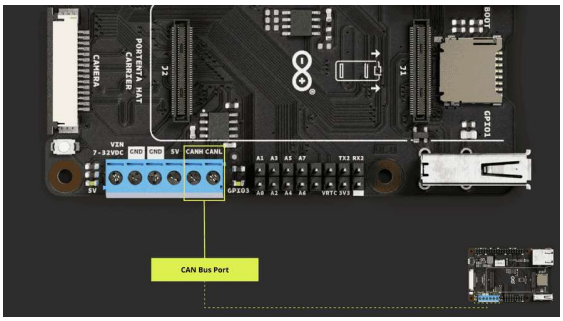
```
// The target device's I2C address
byte deviceAddress = 0x1;

// The number of bytes to read
int numBytes = 2;

// Request data from the target device
Wire.requestFrom(deviceAddress, numBytes);

// Read while there is data available
while (Wire.available()) {
  byte data = Wire.read();
}
```

CAN Bus

The CAN bus, short for Controller Area Network bus, is a resilient communication protocol created by Bosch® in the 1980s for vehicles. It lets microcontrollers and devices interact without a central computer. Using a multi-master model, any system device can send data when the bus is available.

This approach ensures system continuity even if one device fails and is especially effective in electrically noisy settings like in vehicles, where various devices need reliable communication

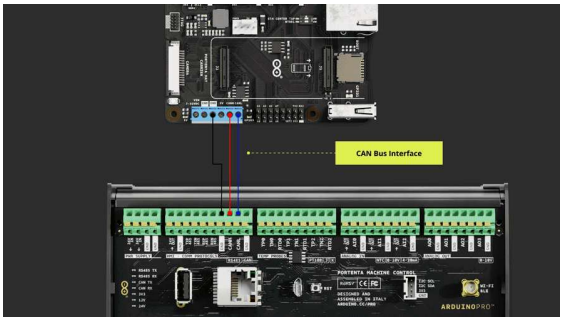
The Portenta Hat Carrier is equipped with CAN bus communication capabilities, powered by the TJA1049 module, and a high-speed CAN FD transceiver. With this, developers can leverage the robustness and efficiency of CAN communication in their projects.



Portenta Hat Carrier CAN Bus Port

Since the CAN bus pins are integrated within the High-Density connectors, they are conveniently accessible on the carrier through the screw terminal. This provides flexibility in connection, allowing developers to design and troubleshoot their systems easily.

Pin number	Silkscreen	High-Density Pin	Interface
5	CANH	J1-49 (Through U1)	CAN BUS - CANH
6	CANL	J1-51 (Through U1)	CAN BUS - CANL



Portenta Hat Carrier CAN Bus Interface Connection Example

For stable CAN bus communication, it is recommended to install a 120 Ω termination resistor between CANH and CANL lines.

Using Linux

For the Portenta X8, when you have admin (root) access, you can execute the following commands within the shell to control the CAN bus protocol. The CAN transceiver can be enabled using the

```
echo 164 > /sys/class/gpio/export && echo out >
```

This command sequence activates the CAN transceiver. It does so by exporting *GPIO 164*, setting its direction to "`out`", and then writing a value of "`0`" to it. Writing 0 as a value to GPIO 164 means that it will set the GPIO to a LOW state.

For Portenta X8, it is possible to use the following commands:

```
sudo modprobe can-dev
```

The necessary modules for CAN (Controller Area Network) support on the Portenta X8 are loaded. The `can-dev` module is added to the system configuration, after which the system is rebooted to apply the changes.

```
echo "can-dev" | sudo tee > /etc/modules-load.d/
sudo systemctl reboot
```

Within the Portenta X8's shell, Docker containers offer a streamlined environment for specific tasks, such as command-based CAN bus operations. The *cansend* command is one such utility that facilitates sending CAN frames. The command to issue such task is as follows:

```
cansend
```

And as an example, if you need to send a specific CAN frame, the *cansend* command can be used to achieve this task. The command follows the format:

```
cansend <CAN Interface [can0 | can1]> <CAN ID>#<
```

`<CAN Interface [can0 | can1]>`: defines the CAN interface that the Portenta X8 will use with the Portenta Hat Carrier.

`<CAN ID>`: is the identifier of the message and is used for message prioritization. The identifier can be in 11-bit or 29-bit format.

For instance, the following command example would send a CAN message on the `can0` interface with an ID of `123`, using an 11-bit identifier, and a data payload of `DEADBEEF`:

```
cansend can0 123#DEADBEEF
```

While the following command example with a 29-bit identifier would send a CAN message with an extended ID of `1F334455` and a data payload of `1122334455667788`.

```
cansend can0 1F334455#1122334455667788
```

This command sends a message with an extended CAN ID and an 8-byte payload.

To use the *cansend* command, it is crucial to set up the appropriate environment. First, clone the following container repository

```
git clone https://github.com/pika-spark/pika-spa
```

Navigate to the *can-utils-sh* directory:

```
cd pika-spark-containers/can-utils-sh
```

Build the Docker container:

```
./docker-build.sh
```

Run the Docker container with the desired CAN interface and bitrate:

```
sudo ./docker-run.sh can0 | can1 [bitrate]
```

Moreover, if your goal is to monitor and dump all received CAN

the container repository ready with its components, navigate to the *candump* directory:

```
cd pika-spark-containers/candump
```

Build the Docker container:

```
./docker-build.sh
```

Run the Docker container with the desired CAN interface and bitrate:

```
sudo ./docker-run.sh can0 | can1 [bitrate]
```

As an example, the command can be structured as follows:

```
sudo ./docker-run.sh can0 250000
```

For more information regarding this container utility, please check *can-utils-sh* and ***candump***.

The list provided offers a quick reference for various `bustype` parameters supported by *python-can* library.

- **socketcan**: For the SocketCAN interface, which is native to Linux.
- **virtual**: Creates a virtual CAN bus, useful for testing purposes when you do not have actual CAN hardware.
- **pcan**: For Peak-System PCAN-USB adapters.
- **canalystii**: For the Canalyst II interface.
- **kvaser**: For Kvaser CAN interfaces.
- **systec**: For SYS TEC electronic interfaces.
- **vector**: For Vector hardware using the XL Driver Library.
- **usb2can**: For the 8devices USB2CAN.
- **ixxat**: For IXXAT hardware using the VCI driver.
- **nican**: For National Instruments CAN hardware.
- **iscan**: For the Intrepid Control Systems (ICS) neoVI.

Each bustype corresponds to different CAN interfaces or devices, ranging from the native Linux *SocketCAN* interface to specific hardware devices like *Kvaser*, *Vector*, and more.

standard and extended CAN messages respectively. The `main()` function creates a CAN bus instance with a 'virtual' bus type for demonstration purposes and sends standard and extended CAN messages in a loop.

```
import can
import time

def send_standard_can_message(channel, messag
  msg = can.Message(arbitration_id=message_id
  channel.send(msg)

def send_extended_can_message(channel, messag
  msg = can.Message(arbitration_id=message_id
  channel.send(msg)

def main():
  # Assuming you're using a virtual channel f
  # If you're using real hardware like the So
  bus = can.interface.Bus(channel='virtual', 

  while True:
    print("Sending packet ... ", end="")
    send_standard_can_message(bus, 0x12, [ord
    print("done")

    time.sleep(1)

    print("Sending extended packet ... ", end
    send_extended_can_message(bus, 0xabcdef, 
    print("done")

    time.sleep(1)
```

Continuing Python® script defines functions to receive and print incoming CAN messages. The receive_can_messages function continuously listens for CAN messages and calls print_received_message to display the details of the received message, such as whether it is an extended message or a remote transmission request (RTR) and its data.

```
import can

def receive_can_messages(bus):
    while True:
        message = bus.recv()  # Blocks until a
        print_received_message(message)

def print_received_message(message):
    print("Received ", end="")

    if message.is_extended_id:
        print("extended ", end="")

    if message.is_remote_frame:
        print("RTR ", end="")

    print("packet with id 0x{:X}".format(mess
    if message.is_remote_frame:
        print(" and requested length {}".forma
    else:
        print(" and length {}".format(len(mes

        # Only print packet data for non-RTR p
        for byte in message.data:
            print(chr(byte), end="")
        print()
```

The `main()` function initializes the CAN bus with a 'virtual' channel for example demonstration and starts the message listening process.

Using Arduino IDE

For users working with the Portenta H7 or Portenta C33, the following simple examples can be used to test the CAN bus protocol's capabilities.

The *CAN Read* example for Portenta H7/C33 starts CAN communication at a rate of *250 kbps* and continuously listens for incoming messages, displaying such information upon receipt.

```ino
#include <Arduino_CAN.h>

/**************************************************************
 * SETUP/LOOP
 **************************************************************/

void setup()
{
  Serial.begin(115200);
  while (!Serial) { }

  if (!CAN.begin(CanBitRate::BR_250k))
  {
    Serial.println("CAN.begin(...) failed.");
    for (;;) {}
  }
}

void loop()
{
  if (CAN.available())
  {
    CanMsg const msg = CAN.read();
    Serial.println(msg);
  }
}
```

The *CAN Write* example, also set at *250 kbps*, builds and sends a specific message format. This message includes a fixed preamble followed by an incrementing counter value that updates with each loop iteration.

```
/**************************************************************************************
 * INCLUDE
 **************************************************************************************/

#include <Arduino_CAN.h>

/**************************************************************************************
 * CONSTANTS
 **************************************************************************************/

static uint32_t const CAN_ID = 0x20;

/**************************************************************************************
 * SETUP/LOOP
 **************************************************************************************/

void setup()
{
  Serial.begin(115200);
  while (!Serial) { }

  if (!CAN.begin(CanBitRate::BR_250k))
  {
    Serial.println("CAN.begin(...) failed.");
    for (;;) {}
  }
}
```

UART

The Portenta Hat Carrier supports UART communication. The pins used in the Portenta Hat Carrier for the UART communication protocol are the following:

Pin number	Silkscreen	Portenta HD Standard Pin	High-Density Pin	Interface
40-Pin Header				
8	TX3	SERIAL3_TX	J2-25	UART 3 TX
10	RX3	SERIAL3_RX	J2-27	UART 3 RX
29	RX1	SERIAL1_RX	J1-35	UART 1 RX
32	TX1	SERIAL1_TX	J1-33	UART 1 TX
16-Pin Header				
14	TX2	SERIAL2_TX	J2-26	UART 2 TX

Pin number	Silkscreen	Portenta HD Standard Pin	High-Density Pin	Interface
16	RX2	SERIAL2_RX	J2-28	UART 2 RX

Please, refer to the board pinout section of the user manual to find them on the board. The UART pins can be used through the built-in (Serial) library functions.

Using Linux

For the Portenta X8, when you have admin (root) access, you can execute the command `ls /dev/ttyUSB* /dev/ttyACM* /dev/ttymxc*` within the shell to list available serial ports in Linux. Typically, USB serial devices could appear as */dev/ttyUSBx*, */dev/ttyACMx*, or */dev/ttymxcx*.

```
ls /dev/ttyUSB* /dev/ttyACM* /dev/ttymxc*
```

```
/dev/ttymxc2   // Something similar
```

The output */dev/ttymxc2* is an example of a potential serial device you might find on Portenta X8.

Following Python® script uses the *pyserial* library to communicate with devices over UART. It defines the `processData` function which prints the received data. You can modify this function based on your application's needs.

```
import serial
import time

# Define the processData function (you'll nee
def processData(data):
    print("Received:", data)  # For now, just

# Set up the serial port
ser = serial.Serial('/dev/ttymxc2', 9600)  # I

incoming = ""

while True:
  # Check for available data and read individ
  while ser.in_waiting:
    c = ser.read().decode('utf-8')  # Read a

    # Check if the character is a newline (li
    if c == '\n':
      # Process the received data
      processData(incoming)

      # Clear the incoming data string for th
      incoming = ""
    else:
      # Add the character to the incoming dat
      incoming += c
```

The script sets up a serial connection on port */dev/ttymxc2* at a baud rate of *9600*. It then continuously checks for incoming data. When a newline character (`\n`) is detected, indicating the end of a message, the script processes the accumulated data and then resets the data string to be ready for the next incoming message.

The `time.sleep(0.002)` line adds a slight delay, ensuring data has enough time to buffer, similar to using `delay(2);` in Arduino.

Using Arduino IDE

For Portenta H7 or C33, the following examples can be used to test UART communication. For a proper UART communication, the baud rate (bits per second) must be set within the `setup()` function.

```
// Start UART communication at 9600 baud
Serial.begin(9600);
```

With the `Serial1.available()` function and the `Serial1.read()` function, you can read incoming data by using a `while()` loop to repeatedly check for available data and read

the code below processes the input and stores the incoming characters in a String variable.:

Using the `Serial1.available()` method alongside the `Serial1.read()` method allows you to read incoming data. The technique involves using a `while()` loop to consistently check for available data and then read individual characters.

Upon receiving a line-ending character, the code processes the accumulated characters and stores them in a *String* variable. The relevant section of the code is presented below:

```
void loop() {
  while (Serial1.available()) {
    delay(2);
    char c = Serial1.read();
    if (c == '\n') {
      processData(incoming);
      incoming = "";
    } else {
      incoming += c;
    }
  }
}

void processData(String data) {
  Serial.println("Received: " + data);  // Prir
}
```

Consequently, the following provides a complete example illustrating the reception of incoming data via UART:

```
String incoming = "";

void setup() {
  Serial1.begin(9600);  // For communication wi
  Serial.begin(9600);   // For debugging over U
}

void loop() {
  while (Serial1.available()) {
    delay(2);
    char c = Serial1.read();
    if (c == '\n') {
      processData(incoming);
      incoming = "";
    } else {
      incoming += c;
    }
  }
}

void processData(String data) {
  Serial.println("Received: " + data);  // Prir
}
```

For transmitting data over UART, which can complement the receiving board as depicted above, refer to the ensuing example:

```
void setup() {
  Serial1.begin(9600);
}

void loop() {
  Serial1.println("Hello from Portenta!");
  delay(1000);
}
```

Using these codes, you should observe the message `"Hello from Portenta!"` being transmitted to the receiving Portenta board when coupled with a Portenta Hat Carrier.

The `Serial.write()` method allows you to send data over UART (Universal Asynchronous Receiver-Transmitter). This method is used when you want to transmit raw bytes or send a byte array.

```
// Transmit the string "Hello world!
Serial.write("Hello world!");
```

Code snippet above sends the string "`Hello world!`" to another

The following methods are used to send strings over UART.

```
// Transmit the string "Hello world!"
Serial.print("Hello world!");

// Transmit the string "Hello world!" followed b
Serial.println("Hello world!");
```

The difference between the two is that `Serial.println()` sends a newline character (`\n`) after sending the string, thereby moving the cursor to the next line, whereas `Serial.print()` does not.

Support

If you encounter any issues or have questions while working with the Portenta Hat Carrier, we provide various support resources to help you find answers and solutions.

Help Center

Explore our Help Center, which offers a comprehensive collection of articles and guides for the Portenta Hat Carrier. The Arduino Help Center is designed to provide in-depth technical assistance and help you make the most of your device.

Portenta Hat Carrier help center page

Forum

Join our community forum to connect with other Portenta Hat Carrier users, share your experiences, and ask questions. The forum is an excellent place to learn from others, discuss issues, and discover new ideas and projects related to the Portenta Hat Carrier.

Portenta Hat Carrier category in the Arduino Forum

Contact Us

Please get in touch with our support team if you need personalized assistance or have questions not covered by the help and support resources described before. We're happy to help you with any issues or inquiries about the Portenta Hat Carrier.

Contact us page

Suggest changes

The content on docs.arduino.cc is facilitated through a public GitHub repository. If you see anything wrong, you can edit this page here.

Need support?

Help Center
Ask the Arduino Forum
Discover Arduino Discord

License

Was this article helpful?